

TRƯỜNG ĐẠI HỌC HÒA BÌNH
KHOA CÔNG NGHỆ THÔNG TIN & ĐIỆN TỬ VIỄN THÔNG



BÀI TẬP LỚN
MÔN: PHÁT TRIỂN ỨNG DỤNG CHO CÁC THIẾT BỊ
Đề tài: Xây dựng ứng dụng bán hàng online SaleApp trên nền
tảng Android

Sinh viên thực hiện	Kiều Minh Huyền
Mã sinh viên	523DPT1005
Lớp	: 523DPT
Giáo viên hướng dẫn	: Th.S Lê Thị Lương

HÀ NỘI – 2026

TRƯỜNG ĐẠI HỌC HÒA BÌNH
KHOA CÔNG NGHỆ THÔNG TIN & ĐIỆN TỬ VIỄN THÔNG



BÀI TẬP LỚN

MÔN:

Đề tài: Xây dựng ứng dụng bán hàng online SaleApp trên nền tảng Android

STT	Mã sinh viên	Họ và tên	Ngày sinh	Điểm	
				Bảng số	Bảng chữ
1	523DPT1005	Kiều Minh Huyền	20/06/2005		

Cán bộ chấm thi 1

Cán bộ chấm thi 2

HÀ NỘI – 2026

LỜI CẢM ƠN

Để hoàn thành được bài tập lớn này, em xin gửi lời cảm ơn chân thành đến Th.S Lê Thị Lương, giảng viên hướng dẫn đã tận tình chỉ dẫn, góp ý trong suốt quá trình thực hiện đề tài.

Em cũng xin cảm ơn Khoa Công nghệ Thông tin & Điện tử Viễn thông, Trường Đại học Hòa Bình đã tạo điều kiện về tài liệu, cơ sở vật chất và môi trường học tập thuận lợi.

Do thời gian và kinh nghiệm còn hạn chế, báo cáo không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy cô để có thể hoàn thiện hơn trong những lần thực hiện tiếp theo.

LỜI MỞ ĐẦU

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ thông tin và sự phổ biến của các thiết bị di động thông minh đã làm thay đổi đáng kể thói quen mua sắm của người tiêu dùng. Thay vì thực hiện các hoạt động mua bán theo phương thức truyền thống, người dùng ngày càng có xu hướng sử dụng các nền tảng trực tuyến để tìm kiếm thông tin, lựa chọn sản phẩm và thực hiện giao dịch một cách nhanh chóng, thuận tiện. Đặc biệt, với sự phát triển của điện thoại thông minh, các ứng dụng bán hàng trên nền tảng di động đang trở thành một phương thức quan trọng giúp kết nối giữa doanh nghiệp và khách hàng.

Bên cạnh nhu cầu thực tế của thị trường, việc nghiên cứu và xây dựng ứng dụng di động cũng là một nội dung quan trọng đối với sinh viên ngành Công nghệ thông tin. Quá trình phát triển một ứng dụng Android yêu cầu người thực hiện vận dụng nhiều kiến thức như phân tích hệ thống, thiết kế giao diện người dùng, lập trình xử lý chức năng và quản lý dữ liệu. Đây là cơ hội để sinh viên áp dụng những kiến thức đã học vào việc xây dựng một sản phẩm phần mềm có tính thực tiễn.

Xuất phát từ những lý do trên, đề tài “Xây dựng ứng dụng bán hàng online SaleApp trên nền tảng Android” được lựa chọn nhằm nghiên cứu và xây dựng một ứng dụng bán hàng trên thiết bị di động. SaleApp là tên gọi do em đặt cho ứng dụng trong phạm vi đề tài này, tập trung vào việc kinh doanh các sản phẩm công nghệ như điện thoại, laptop, tai nghe và các phụ kiện đi kèm. Đây là nhóm sản phẩm có nhu cầu tìm kiếm thông tin và mua sắm trực tuyến cao, đồng thời phù hợp để mô phỏng một hệ thống bán hàng thực tế với các chức năng như quản lý sản phẩm, xem chi tiết sản phẩm, thêm vào giỏ hàng và đặt hàng.

Mục tiêu của đề tài là thiết kế và xây dựng giao diện một ứng dụng bán hàng trực tuyến trên nền tảng Android, mô phỏng quy trình mua sắm cơ bản của một hệ thống thương mại điện tử. Ứng dụng được xây dựng nhằm hỗ trợ người dùng thực hiện các thao tác như đăng nhập, đăng ký tài khoản, xem danh sách sản phẩm, xem thông tin chi tiết, thêm sản phẩm vào giỏ hàng và thực hiện đặt hàng.

MỤC LỤC

CHƯƠNG 1. CƠ SỞ LÝ LUẬN

1.1. Tổng quan về nền tảng Android

1.1.1. Giới thiệu hệ điều hành Android

Android là hệ điều hành dành cho thiết bị di động, ban đầu do công ty Android Inc. phát triển từ năm 2003 và được Google mua lại vào năm 2005. Phiên bản chính thức đầu tiên ra mắt năm 2008, mở đầu cho một trong những hệ điều hành di động phổ biến nhất hiện nay. Android được xây dựng trên dự án mã nguồn mở Android Open Source Project (AOSP), nhờ vậy nhiều nhà sản xuất có thể tùy chỉnh và tích hợp hệ điều hành này vào các dòng thiết bị khác nhau, từ phổ thông đến cao cấp.

Sở dĩ Android giữ được vị trí dẫn đầu về số lượng người dùng là nhờ ba yếu tố: khả năng tương thích rộng với nhiều thiết bị, kho ứng dụng phong phú trên Google Play, và môi trường phát triển khá thân thiện với lập trình viên, đặc biệt là người mới bắt đầu.

1.1.2. Kiến trúc cơ bản của một ứng dụng Android

Một ứng dụng Android không chạy như một khối duy nhất mà được tổ chức từ nhiều thành phần (component) có vai trò riêng. Trong đó, Activity là thành phần quen thuộc nhất, đại diện cho một màn hình cụ thể trong ứng dụng, ví dụ màn hình đăng nhập, màn hình danh sách sản phẩm, màn hình giỏ hàng đều là các Activity tách biệt. Mỗi Activity chịu trách nhiệm hiển thị giao diện và xử lý các thao tác của người dùng diễn ra trên màn hình đó.

Việc tách ứng dụng thành nhiều Activity giúp em dễ quản lý hơn khi lập trình: mỗi màn hình là một file riêng, sửa màn hình này không ảnh hưởng đến màn hình khác. Khi người dùng di chuyển giữa các màn hình chẳng hạn từ danh sách sản phẩm bấm vào xem chi tiết Android dùng đối tượng Intent để thực hiện việc chuyển Activity này.

Bên cạnh Activity, phần giao diện hiển thị của từng màn hình được định nghĩa thông qua Layout, viết dưới dạng file XML. File XML này quy định những thành phần nào xuất hiện trên màn hình, sắp xếp ở đâu, kích thước ra sao. Cách tổ chức này giúp tách riêng phần giao diện (XML) và phần xử lý logic (Kotlin), nhờ đó khi cần đổi giao diện, em chỉ cần sửa file layout mà không phải đụng vào code xử lý phía sau.

1.1.3. Ưu điểm của Android khi xây dựng ứng dụng bán hàng

Với một đề tài mô phỏng ứng dụng bán hàng như thế này, Android có vài điểm thuận lợi rõ ràng. Thứ nhất, lượng người dùng Android rất lớn nên về mặt thực tế, một

ứng dụng bán hàng triển khai trên Android có khả năng tiếp cận khách hàng tốt hơn so với nhiều nền tảng khác.

Thứ hai, công cụ phát triển là Android Studio, được cung cấp miễn phí, đi kèm khá nhiều tính năng hỗ trợ như thiết kế giao diện kéo-thả, giả lập thiết bị, công cụ debug. Điều này giúp giảm đáng kể chi phí học tập và triển khai, vốn là yếu tố quan trọng đối với một bài tập lớn của sinh viên.

Thứ ba, hệ sinh thái Android hỗ trợ nhiều thư viện và công nghệ mở rộng, nên nếu sau này muốn phát triển thêm (ví dụ thêm chức năng thanh toán thật, kết nối server), việc tích hợp cũng không quá phức tạp. Đây là lý do em quyết định chọn Android cho đề tài “Xây dựng ứng dụng bán hàng online SaleApp trên nền tảng Android” vừa khả thi trong thời gian làm bài tập lớn, vừa có thể mở rộng nếu cần.

1.2. Công cụ phát triển: Android Studio

Đối với đề tài này, Android Studio được em sử dụng làm môi trường chính, xuyên suốt từ lúc thiết kế giao diện, viết code xử lý, đến lúc kiểm thử từng chức năng.

1.2.1. Giới thiệu Android Studio

Android Studio là IDE (môi trường phát triển tích hợp) chính thức do Google phát hành, xây dựng trên nền tảng IntelliJ IDEA. Điểm em thấy tiện nhất khi dùng công cụ này là không cần chuyển qua nhiều phần mềm khác nhau, từ tạo project, thiết kế layout, viết code, quản lý thư viện, đến chạy thử ứng dụng đều thực hiện được ngay trong một môi trường.

Ngoài ra, Android Studio còn hỗ trợ gợi ý code tự động, báo lỗi cú pháp ngay khi đang gõ, và cho phép giả lập nhiều loại thiết bị khác nhau để kiểm tra hiển thị. Với một người mới làm quen với Android như em, những tính năng này giúp giảm khá nhiều thời gian dò lỗi so với việc lập trình hoàn toàn thủ công.

Toàn bộ các màn hình của ứng dụng bán hàng trong đề tài đăng nhập, đăng ký, trang chủ, danh sách sản phẩm, chi tiết sản phẩm, giỏ hàng, thanh toán đều được em thiết kế và kiểm thử trực tiếp trên Android Studio.

1.2.2. Các thành phần chính của Android Studio được sử dụng trong đề tài

Layout Editor

Layout Editor là công cụ thiết kế giao diện trực quan tích hợp sẵn trong Android Studio, cho phép vừa kéo-thả thành phần giao diện, vừa chỉnh sửa trực tiếp mã XML và

xem trước kết quả song song. Em dùng công cụ này để dựng layout cho các màn hình chính của ứng dụng như đăng nhập, đăng ký, danh sách và chi tiết sản phẩm, giỏ hàng — nhờ xem trước được giao diện ngay khi chỉnh sửa XML, việc canh chỉnh bố cục diễn ra nhanh hơn nhiều so với chỉ đọc code XML thuần.

Emulator

Vì không phải lúc nào cũng có thiết bị Android thật trong tay, Emulator (máy giả lập) tích hợp trong Android Studio là công cụ em dùng thường xuyên nhất trong quá trình làm đồ án. Công cụ này mô phỏng khá đầy đủ một thiết bị Android thật, từ hệ điều hành, độ phân giải màn hình, đến các thao tác chạm, cuộn. Em chủ yếu dùng Emulator để kiểm tra lại từng chức năng sau khi code xong, ví dụ luồng chuyển màn hình khi đăng nhập, thao tác thêm sản phẩm vào giỏ hàng, trước khi chuyển sang thử trên thiết bị thật.

Logcat

Trong lúc lập trình, việc gặp lỗi runtime là điều không tránh khỏi, đặc biệt khi ứng dụng có nhiều màn hình liên kết với nhau. Logcat là công cụ hiển thị log hệ thống theo thời gian thực, giúp em xác định được lỗi xảy ra ở đâu, do dòng code nào gây ra. Trong quá trình làm đề tài, em dùng Logcat khá nhiều để debug các lỗi liên quan đến truy vấn SQLite và lỗi null khi xử lý dữ liệu giữa các Activity, đây cũng là công cụ giúp em tiết kiệm thời gian sửa lỗi đáng kể so với việc chỉ đoán nguyên nhân.

Nhìn chung, ba công cụ trên đã hỗ trợ em xuyên suốt quá trình làm đồ án, từ giai đoạn thiết kế giao diện ban đầu đến lúc hoàn thiện và kiểm thử ứng dụng.

Tài liệu tham khảo mục 1.1, 1.2

- [1] Google Android Developers, Build your first app. Nguồn: <https://developer.android.com/>
- [2] Google Android Developers, Meet Android Studio. Nguồn: <https://developer.android.com/studio>
- [3] Google Android Developers, Run apps on the Android Emulator. Nguồn: <https://developer.android.com/studio/run/emulator>

1.3. Ngôn ngữ và công nghệ sử dụng

Ba công nghệ chính được em sử dụng trong đề tài là XML cho thiết kế giao diện, Kotlin cho xử lý logic, và SQLite cho lưu trữ dữ liệu cục bộ. Mỗi công nghệ đảm nhận một vai trò riêng nhưng phối hợp chặt chẽ với nhau: XML quy định giao diện hiển thị,

Kotlin xử lý các thao tác và liên kết các thành phần, còn SQLite lưu và truy xuất dữ liệu phục vụ cho toàn bộ hệ thống.

1.3.1. XML trong thiết kế giao diện

XML (eXtensible Markup Language) là ngôn ngữ đánh dấu dùng để mô tả cấu trúc và bố cục giao diện trong Android. Khác với code xử lý logic, XML chỉ tập trung định nghĩa cách các thành phần hiển thị trên màn hình không chứa logic xử lý.

Trong Android, mỗi màn hình giao diện thường tương ứng với một file XML đặt trong thư mục `res/layout`. Trong file này, em khai báo các thành phần như `TextView`, `ImageView`, `Button`, `EditText`, `RecyclerView`, kết hợp với các `Layout` để tạo bố cục hoàn chỉnh cho từng màn hình.

Lợi ích lớn nhất khi dùng XML là tách biệt được phần giao diện và phần xử lý chức năng khi cần đổi màu, đổi kích thước hay bố cục một màn hình, em chỉ cần sửa file XML tương ứng mà không đụng đến code Kotlin xử lý phía sau, hạn chế việc sửa nhầm hoặc gây lỗi không liên quan. Trong đề tài, toàn bộ giao diện của ứng dụng từ trang đăng nhập đến chi tiết sản phẩm, giỏ hàng đều được xây dựng bằng các file layout XML, giúp các màn hình giữ được sự thống nhất về cách trình bày.

1.3.2. Vai trò của Kotlin trong việc xử lý logic và liên kết giao diện

Nếu XML lo phần “nhìn thấy được” thì Kotlin đảm nhận phần “xử lý phía sau” của ứng dụng. Kotlin là ngôn ngữ lập trình chính được em dùng để viết logic chương trình, xử lý sự kiện người dùng (như bấm nút, nhập liệu) và điều khiển hoạt động giữa các thành phần. So với Java, cú pháp Kotlin ngắn gọn hơn khá nhiều, đồng thời có cơ chế kiểm tra null an toàn ngay từ khi viết code (null safety), giúp em hạn chế được một số lỗi `NullPointerException` thường gặp khi xử lý dữ liệu lấy từ SQLite hoặc truyền giữa các Activity.

Cụ thể, mỗi khi người dùng thực hiện một hành động trên giao diện ví dụ bấm nút “Đăng nhập” phần code Kotlin sẽ tiếp nhận sự kiện đó, kiểm tra thông tin tài khoản với dữ liệu trong SQLite, rồi quyết định chuyển sang màn hình tiếp theo hoặc hiển thị thông báo lỗi nếu thông tin không hợp lệ. Tương tự, các thao tác như chọn sản phẩm, thêm vào giỏ hàng, tạo đơn hàng trong ứng dụng đều được xử lý theo cách này.

Ngoài xử lý sự kiện, Kotlin còn quản lý việc chuyển đổi giữa các màn hình thông qua Activity và Intent, giúp người dùng di chuyển đúng luồng sử dụng đã thiết kế (ví dụ: không thể vào trang thanh toán nếu chưa đăng nhập). Việc tách riêng XML (giao

diện) và Kotlin (xử lý) cũng giúp em dễ kiểm soát mã nguồn hơn trong quá trình làm đồ án khi gặp lỗi, em có thể nhanh xác định lỗi đó nằm ở phần hiển thị hay phần xử lý.

1.3.3. SQLite lưu trữ dữ liệu cục bộ cho ứng dụng

SQLite là hệ quản trị cơ sở dữ liệu quan hệ được tích hợp sẵn trong Android, cho phép ứng dụng lưu và quản lý dữ liệu trực tiếp trên thiết bị mà không cần đến server bên ngoài. Khác với các hệ quản trị cơ sở dữ liệu cần cài đặt máy chủ riêng, toàn bộ dữ liệu của SQLite được lưu trong một file duy nhất trên thiết bị, nên việc triển khai cũng đơn giản hơn nhiều.

Đây là lựa chọn phù hợp với quy mô của đề tài: vì là ứng dụng mô phỏng phục vụ mục đích học tập, không cần đến hạ tầng server thật, SQLite đáp ứng đủ cho các nhu cầu lưu trữ cơ bản như thông tin tài khoản người dùng, danh sách sản phẩm, dữ liệu giỏ hàng và đơn hàng. Trong ứng dụng, em dùng SQLite để thực hiện các thao tác thêm, sửa, xóa, tìm kiếm dữ liệu phục vụ cho từng chức năng chẳng hạn lưu thông tin đăng ký, kiểm tra tài khoản khi đăng nhập, hoặc cập nhật số lượng sản phẩm trong giỏ hàng.

Việc chọn SQLite giúp em triển khai được một mô hình ứng dụng bán hàng tương đối đầy đủ mà không cần xây dựng hệ thống backend riêng, điều này phù hợp với mục tiêu của bài tập lớn là tập trung vào thiết kế giao diện và luồng hoạt động của ứng dụng, hơn là xây dựng một hệ thống thương mại điện tử hoàn chỉnh.

Tài liệu tham khảo mục 1.3

- [1] Google Android Developers, Build your first app. Nguồn: <https://developer.android.com/>
- [2] SQLite Documentation, About SQLite. Nguồn: <https://www.sqlite.org/docs.html>

1.4. Các thành phần giao diện cơ bản

Giao diện là yếu tố quyết định trải nghiệm sử dụng của một ứng dụng, một ứng dụng có đầy đủ chức năng nhưng bố cục lộn xộn, khó nhìn thì vẫn khó dùng. Để xây dựng giao diện cho ứng dụng bán hàng, Android cung cấp hai nhóm thành phần chính: Layout (tổ chức vị trí, sắp xếp) và Widget (hiển thị, nhận tương tác từ người dùng).

Trong đề tài, các Layout được sử dụng gồm LinearLayout, ScrollView, CardView; các Widget gồm TextView, EditText, Button, ImageView. Phần dưới đây trình bày cụ thể vai trò của từng thành phần.

1.4.1. Các Layout dùng để bố trí giao diện

LinearLayout sắp xếp các thành phần theo một hướng duy nhất là hướng dọc (vertical) hoặc ngang (horizontal). Em dùng LinearLayout cho những khu vực có cấu trúc đơn giản, cần xếp theo trình tự rõ ràng, ví dụ khu vực nhập thông tin đăng nhập (ô tài khoản, ô mật khẩu, nút đăng nhập xếp theo chiều dọc) hoặc nhóm các nút chức năng.

ScrollView cho phép cuộn nội dung khi màn hình không đủ chỗ hiển thị hết. Thành phần này em dùng chủ yếu ở màn hình chi tiết sản phẩm nơi cần hiển thị nhiều thông tin cùng lúc như hình ảnh, tên sản phẩm, thông số kỹ thuật, giá và mô tả, vốn thường dài hơn một màn hình.

CardView tạo khung hiển thị dạng thẻ, có bo góc và đổ bóng nhẹ, giúp giao diện trông hiện đại và dễ phân biệt từng nhóm thông tin. Trong ứng dụng, mỗi sản phẩm trong danh sách được hiển thị dưới dạng một CardView riêng, chứa hình ảnh, tên và giá sản phẩm cách này giúp người dùng dễ quét mắt qua danh sách và so sánh sản phẩm.

1.4.2. Các Widget sử dụng trong giao diện ứng dụng

TextView dùng để hiển thị văn bản tên sản phẩm, giá, mô tả, tiêu đề màn hình. Đây là widget xuất hiện nhiều nhất trong ứng dụng vì hầu hết thông tin sản phẩm đều cần thể hiện dưới dạng chữ.

EditText cho phép người dùng nhập dữ liệu, được em dùng ở các màn hình cần nhập thông tin như đăng nhập, đăng ký tài khoản, hoặc nhập địa chỉ giao hàng khi đặt hàng.

Button đại diện cho các nút bấm thực hiện hành động đăng nhập, thêm vào giỏ hàng, đặt hàng, xác nhận thanh toán. Mỗi Button trong ứng dụng đều gắn với một sự kiện xử lý tương ứng ở phần code Kotlin.

ImageView dùng để hiển thị hình ảnh sản phẩm điện thoại, laptop, tai nghe giúp người dùng dễ nhận diện sản phẩm hơn so với chỉ đọc tên và mô tả bằng chữ.

Việc kết hợp các Layout và Widget kể trên giúp ứng dụng có giao diện rõ ràng, dễ thao tác, đồng thời tạo sự thống nhất giữa các màn hình trong toàn bộ hệ thống.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích yêu cầu hệ thống

2.1.1. Mô tả bài toán

Việc mua sắm đồ công nghệ theo phương thức truyền thống còn tồn tại một số hạn chế rõ ràng: khách hàng phải trực tiếp đến cửa hàng, phụ thuộc vào giờ mở cửa và địa điểm, trong khi thông tin sản phẩm không phải lúc nào cũng được trình bày đầy đủ, rõ ràng. Điều này gây bất tiện cho người dùng, đặc biệt khi nhu cầu tìm hiểu kỹ thông số kỹ thuật, so sánh giá hoặc mua hàng ngoài giờ hành chính ngày càng phổ biến.

Một ứng dụng bán đồ công nghệ trên nền tảng Android có thể giải quyết những hạn chế trên bằng cách cung cấp môi trường mua sắm trực tuyến ngay trên thiết bị di động: người dùng chủ động xem danh sách sản phẩm, tra cứu thông tin chi tiết, thêm vào giỏ hàng và đặt hàng bất kể thời gian hay địa điểm.

Xuất phát từ nhu cầu đó, đề tài xây dựng một ứng dụng bán hàng online nhằm mô phỏng hệ thống thương mại điện tử dành cho các sản phẩm công nghệ như điện thoại, laptop, tai nghe và phụ kiện. Ứng dụng sử dụng cơ sở dữ liệu cục bộ SQLite, phù hợp với phạm vi thực nghiệm của bài tập lớn và không yêu cầu kết nối server thực tế.

2.1.2. Mục tiêu hệ thống

Hệ thống hướng đến việc mô phỏng đầy đủ quy trình mua sắm trực tuyến cơ bản trên thiết bị Android, bao gồm các mục tiêu cụ thể sau:

- Xây dựng luồng mua sắm hoàn chỉnh cho người dùng: từ đăng nhập, duyệt sản phẩm, thêm vào giỏ hàng đến đặt hàng và theo dõi lịch sử mua hàng.
- Xây dựng khu vực quản trị cho Admin: cho phép quản lý sản phẩm (thêm/sửa/xóa), theo dõi đơn hàng và quản lý tài khoản người dùng.
- Thiết kế giao diện trực quan, thân thiện, phù hợp với đặc điểm thao tác trên màn hình cảm ứng di động.
- Tổ chức cơ sở dữ liệu SQLite đảm bảo tính toàn vẹn, tránh trùng lặp và hỗ trợ các thao tác CRUD hiệu quả.

2.1.3. Yêu cầu chức năng

Bảng dưới đây tổng hợp các chức năng chính mà hệ thống cần cung cấp, phân loại theo nhóm đối tượng sử dụng:

STT	Chức năng	Mô tả
1	Đăng ký tài khoản	Người dùng tạo tài khoản mới bằng email, họ tên, mật khẩu, số điện thoại và địa chỉ. Hệ thống kiểm tra trùng email trước khi lưu vào CSDL.
2	Đăng nhập / Phân quyền	Xác thực tài khoản và tự động phân quyền: tài khoản User chuyển đến giao diện mua sắm, tài khoản Admin chuyển đến giao diện quản trị.
3	Xem danh sách sản phẩm	Hiển thị sản phẩm dạng lưới hoặc danh sách, hỗ trợ lọc theo danh mục (điện thoại, laptop, tai nghe, phụ kiện...).
4	Xem chi tiết sản phẩm	Hiển thị đầy đủ hình ảnh, tên, giá, mô tả và số lượng tồn kho của sản phẩm được chọn.
5	Quản lý giỏ hàng	Thêm sản phẩm, điều chỉnh số lượng hoặc xóa sản phẩm khỏi giỏ; tự động tính lại tổng tiền.
6	Đặt hàng (mô phỏng)	Xác nhận thông tin nhận hàng, chọn phương thức thanh toán (giả lập), tạo đơn hàng và lưu vào CSDL.
7	Xem lịch sử đơn hàng	Hiển thị danh sách đơn hàng đã đặt kèm trạng thái xử lý; xem chi tiết từng đơn.
8	Quản lý tài khoản cá nhân	Người dùng xem và cập nhật thông tin cá nhân (họ tên, số điện thoại, địa chỉ).
9	Quản lý sản phẩm (Admin)	Admin thêm, sửa, xóa thông tin sản phẩm bao gồm tên, danh mục, giá, số lượng, hình ảnh và mô tả.
10	Quản lý đơn hàng (Admin)	Admin xem danh sách đơn hàng, kiểm tra chi tiết và cập nhật trạng thái xử lý.
11	Quản lý người dùng (Admin)	Admin xem danh sách tài khoản người dùng trong hệ thống.

Bảng 2.1. Yêu cầu chức năng của hệ thống

2.1.4. Yêu cầu phi chức năng

Ngoài các chức năng nghiệp vụ, hệ thống còn cần đáp ứng một số yêu cầu liên quan đến chất lượng phần mềm:

Nhóm yêu cầu	Mô tả cụ thể
Giao diện (UI)	Bố cục rõ ràng, thống nhất màu sắc và font chữ xuyên suốt ứng dụng; các thành phần điều hướng dễ nhận biết.
Khả năng sử dụng (UX)	Người dùng mới có thể thực hiện thao tác mua hàng hoàn chỉnh mà không cần hướng dẫn thêm; luồng thao tác tuần tự, hợp lý.
Hiệu năng	Thao tác chuyển màn hình và hiển thị dữ liệu phản hồi trong thời gian chấp nhận được; không xảy ra treo hay crash khi thao tác bình thường.
Tương thích	Giao diện hiển thị ổn định trên nhiều kích thước màn hình Android phổ biến (từ 5 inch trở lên); tương thích Android 8.0+.
Bảo mật (cơ bản)	Mật khẩu được lưu trữ an toàn; không cho phép truy cập chức năng Admin nếu chưa xác thực đúng vai trò.

Bảng 2.2. Yêu cầu phi chức năng của hệ thống

2.2. Xác định tác nhân và phạm vi hệ thống

Trong một hệ thống phần mềm, tác nhân (Actor) là những đối tượng bên ngoài tương tác trực tiếp với hệ thống nhằm thực hiện một chức năng cụ thể. Việc xác định rõ tác nhân là bước quan trọng để xây dựng Use Case Diagram và phân chia chức năng phù hợp.

Hệ thống bán hàng online trong đề tài có hai tác nhân chính:

a) Người dùng (User)

Là khách hàng sử dụng ứng dụng với mục đích tìm kiếm, lựa chọn và đặt mua sản phẩm công nghệ. Đây là nhóm tác nhân có tần suất tương tác cao nhất và sử dụng toàn bộ luồng mua sắm của ứng dụng. Tài khoản User được tạo thông qua chức năng đăng ký công khai trong ứng dụng.

b) Quản trị viên (Admin)

Là người chịu trách nhiệm quản lý dữ liệu và duy trì hoạt động của hệ thống. Admin không thực hiện thao tác mua hàng mà tập trung vào việc quản lý sản phẩm, theo dõi đơn hàng và quản lý người dùng. Tài khoản Admin không được tạo qua đăng ký thông thường mà được thêm trực tiếp vào cơ sở dữ liệu, đảm bảo kiểm soát quyền truy cập quản trị.

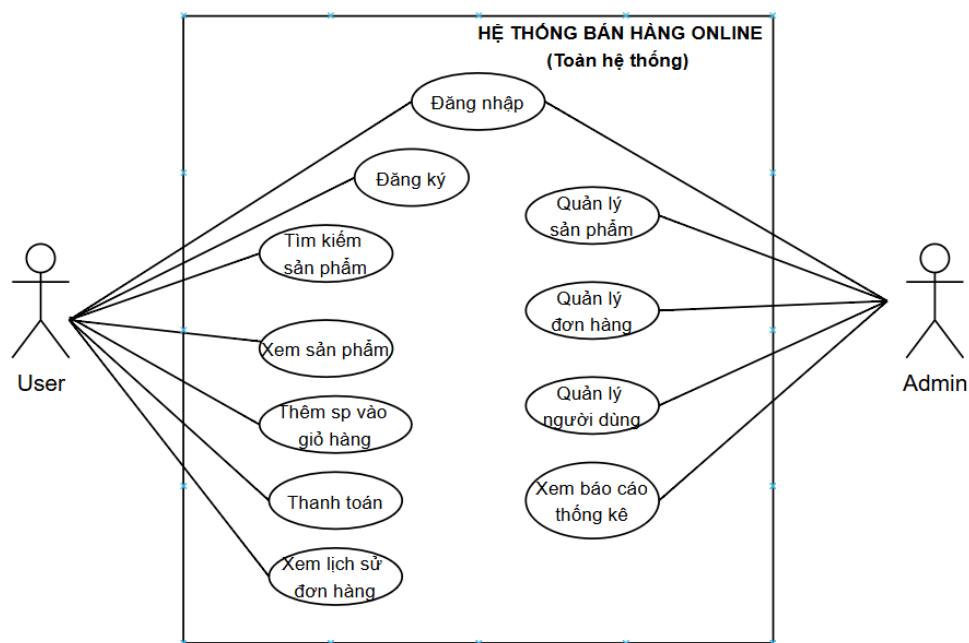
Việc tách biệt hai tác nhân này giúp hệ thống phân quyền rõ ràng: sau khi đăng nhập, ứng dụng kiểm tra loại tài khoản và điều hướng người dùng đến giao diện tương ứng (giao diện mua sắm hoặc giao diện quản trị), đảm bảo mỗi đối tượng chỉ truy cập được các chức năng thuộc quyền hạn của mình.

2.3. Thiết kế Use Case Diagram

Use Case Diagram là công cụ mô hình hóa giúp thể hiện các chức năng mà hệ thống cung cấp và mối quan hệ giữa từng chức năng đó với các tác nhân tương ứng. Sơ đồ Use Case không mô tả cách hệ thống thực hiện chức năng mà chỉ xác định hệ thống làm gì và ai là người sử dụng chức năng đó.

2.3.1. Use Case tổng quát toàn hệ thống

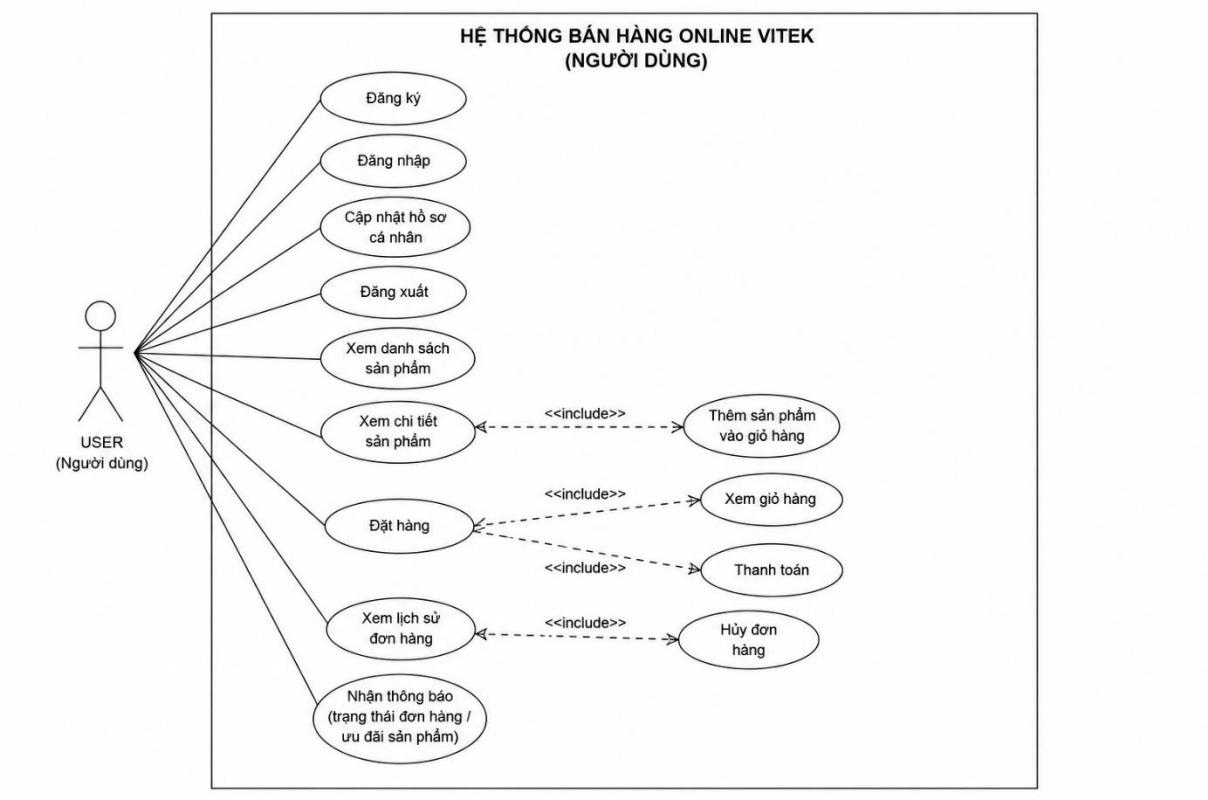
Hệ thống bán hàng online gồm hai tác nhân là Người dùng (User) và Quản trị viên (Admin). User tương tác với các chức năng mua sắm, Admin tương tác với các chức năng quản lý. Hai nhóm chức năng này tách biệt nhau và được phân quyền rõ ràng sau bước đăng nhập.



(Hình 2.2. Sơ đồ Use Case tổng quát hệ thống)

2.3.2. Use Case của User

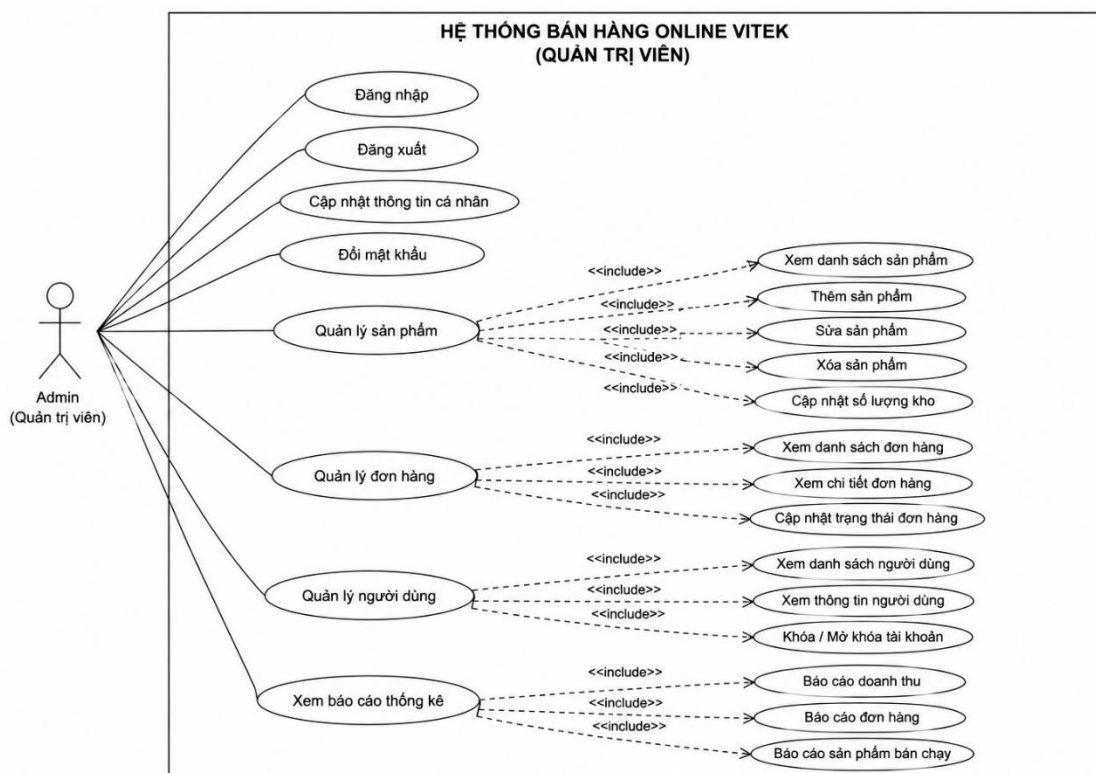
Sơ đồ Use Case của User mô tả toàn bộ tương tác của khách hàng với ứng dụng trong quá trình mua sắm, bao gồm các chức năng từ đăng ký, đăng nhập đến đặt hàng và theo dõi lịch sử.



(Hình 2.3. Sơ đồ Use Case của User)

2.3.3. Use Case của Admin

Sơ đồ Use Case của Admin tập trung vào các chức năng quản trị: đăng nhập vào khu vực quản lý, quản lý sản phẩm (thêm/sửa/xóa), theo dõi và cập nhật trạng thái đơn hàng, cùng với việc xem danh sách người dùng.



(Hình 2.4. Sơ đồ Use Case của Admin)

2.4. Đặc tả Use Case chi tiết

Mỗi Use Case trong bảng đặc tả dưới đây trình bày đầy đủ: tên chức năng, tác nhân tham gia, mô tả ngắn, điều kiện tiên quyết, luồng xử lý chính và kết quả đầu ra. Đây là cơ sở trực tiếp để xây dựng logic xử lý trong từng Activity của ứng dụng Android.

2.4.1. Đăng nhập / Phân quyền

Tên Use Case	Đăng nhập / Phân quyền tài khoản
Tác nhân	User, Admin
Mô tả	Người dùng nhập thông tin tài khoản để truy cập hệ thống. Sau khi xác thực thành công, hệ thống kiểm tra loại tài khoản (User hoặc Admin) và điều hướng đến giao diện phù hợp.
Điều kiện trước	Người dùng đã có tài khoản trong hệ thống; ứng dụng đang chạy bình thường.
Luồng xử lý chính	1. Người dùng nhập email và mật khẩu. 2. Hệ thống kiểm tra thông tin trong CSDL (bảng User và Admin). 3. Nếu thông tin đúng, xác định loại tài khoản. 4. Nếu là User → chuyển đến màn hình Home (mua sắm). 5.

	Nếu là Admin → chuyển đến màn hình Dashboard (quản trị). 6. Nếu sai thông tin → hiển thị thông báo lỗi, yêu cầu nhập lại.
Luồng thay thế	Thông tin sai: hệ thống thông báo lỗi và giữ nguyên màn hình đăng nhập.
Kết quả	Người dùng truy cập được đúng giao diện tương ứng với quyền hạn tài khoản.

2.4.2. Đăng ký tài khoản

Tên Use Case	Đăng ký tài khoản mới
Tác nhân	User (khách chưa có tài khoản)
Mô tả	Cho phép người dùng tạo tài khoản mới để sử dụng các chức năng mua sắm trong ứng dụng.
Điều kiện trước	Người dùng chưa đăng ký hoặc muốn tạo tài khoản bổ sung; email chưa tồn tại trong hệ thống.
Luồng xử lý chính	1. Người dùng chọn "Đăng ký" từ màn hình đăng nhập. 2. Nhập đầy đủ: họ tên, email, mật khẩu, số điện thoại, địa chỉ. 3. Hệ thống kiểm tra tính hợp lệ (không bỏ trống, email đúng định dạng, mật khẩu đủ dài). 4. Kiểm tra email đã tồn tại trong bảng User chưa. 5. Nếu hợp lệ → lưu tài khoản mới vào CSDL → thông báo thành công → chuyển về màn hình đăng nhập.
Luồng thay thế	Email đã tồn tại: hệ thống báo lỗi, không tạo tài khoản trùng.
Kết quả	Tài khoản mới được tạo thành công và có thể dùng để đăng nhập.

2.4.3. Xem danh sách sản phẩm

Tên Use Case	Xem danh sách sản phẩm
Tác nhân	User
Mô tả	Hiển thị toàn bộ sản phẩm đang bán trong hệ thống; hỗ trợ lọc theo danh mục để người dùng dễ tìm kiếm.

Điều kiện trước	Người dùng đã đăng nhập; dữ liệu sản phẩm đã có trong CSDL.
Luồng xử lý chính	1. Người dùng vào màn hình danh sách sản phẩm (từ Home hoặc menu điều hướng). 2. Hệ thống truy vấn bảng Product và Category từ SQLite. 3. Hiển thị sản phẩm dạng danh sách (RecyclerView) với hình ảnh, tên và giá. 4. Người dùng có thể chọn danh mục để lọc hoặc nhấn vào một sản phẩm để xem chi tiết.
Kết quả	Danh sách sản phẩm hiển thị đầy đủ, người dùng có thể chọn sản phẩm tiếp theo.

2.4.4. Xem chi tiết sản phẩm

Tên Use Case	Xem chi tiết sản phẩm
Tác nhân	User
Mô tả	Hiển thị đầy đủ thông tin một sản phẩm để người dùng đánh giá trước khi quyết định mua.
Điều kiện trước	Người dùng đang ở màn hình danh sách và đã chọn một sản phẩm cụ thể.
Luồng xử lý chính	1. Người dùng nhấn vào một sản phẩm trong danh sách. 2. Hệ thống truyền productID sang màn hình chi tiết qua Intent. 3. Truy vấn thông tin sản phẩm từ bảng Product theo productID. 4. Hiển thị hình ảnh, tên, giá, mô tả, số lượng tồn kho. 5. Người dùng có thể nhấn "Thêm vào giỏ hàng".
Kết quả	Người dùng xem được đầy đủ thông tin sản phẩm và có thể tiến hành thêm vào giỏ.

2.4.5. Thêm sản phẩm vào giỏ hàng

Tên Use Case	Thêm sản phẩm vào giỏ hàng
Tác nhân	User
Mô tả	Cho phép người dùng lưu sản phẩm muốn mua vào giỏ hàng để tiếp tục mua sắm hoặc tiến hành đặt hàng sau.

Điều kiện trước	Người dùng đã đăng nhập và đang xem chi tiết sản phẩm.
Luồng xử lý chính	1. Người dùng nhấn nút "Thêm vào giỏ hàng". 2. Hệ thống kiểm tra xem sản phẩm này đã có trong CartItem của user chưa. 3. Nếu chưa có: tạo bản ghi mới trong CartItem với soLuong = 1. 4. Nếu đã có: tăng soLuong lên 1. 5. Cập nhật tongTien trong bảng Cart. 6. Hiển thị thông báo thêm thành công.
Kết quả	Sản phẩm được lưu vào giỏ hàng và sẵn sàng cho bước đặt hàng.

2.4.6. Đặt hàng (mô phỏng thanh toán)

Tên Use Case	Đặt hàng / Thanh toán
Tác nhân	User
Mô tả	Người dùng xác nhận các sản phẩm trong giỏ và tạo đơn hàng. Trong phạm vi đề tài, thanh toán được mô phỏng (chưa tích hợp cổng thanh toán thực tế).
Điều kiện trước	Người dùng đã đăng nhập và giỏ hàng có ít nhất một sản phẩm.
Luồng xử lý chính	1. Người dùng vào màn hình giỏ hàng và nhấn "Đặt hàng". 2. Màn hình Checkout hiển thị: danh sách sản phẩm, địa chỉ nhận, phương thức thanh toán. 3. Người dùng xác nhận thông tin và nhấn "Xác nhận đặt hàng". 4. Hệ thống tạo bản ghi trong bảng Orders (ngày đặt, trạng thái "Đang xử lý", tổng tiền...). 5. Tạo các bản ghi trong bảng OrderDetail (mỗi sản phẩm một dòng). 6. Xóa các CartItem đã đặt hàng. 7. Hiển thị màn hình xác nhận đặt hàng thành công.
Kết quả	Đơn hàng được tạo và lưu vào CSDL; người dùng có thể theo dõi trong lịch sử đơn hàng.

2.4.7. Quản lý sản phẩm (Admin)

Tên Use Case	Quản lý sản phẩm
Tác nhân	Admin

Mô tả	Admin thực hiện đầy đủ thao tác CRUD đối với dữ liệu sản phẩm: xem danh sách, thêm mới, chỉnh sửa và xóa sản phẩm.
Điều kiện trước	Admin đã đăng nhập thành công vào khu vực quản trị.
Luồng xử lý chính	1. Admin truy cập màn hình Quản lý sản phẩm. 2. Hệ thống hiển thị danh sách toàn bộ sản phẩm từ bảng Product. 3a. Thêm: Admin nhập thông tin sản phẩm mới → hệ thống lưu vào bảng Product. 3b. Sửa: Admin chọn sản phẩm → sửa thông tin → lưu cập nhật. 3c. Xóa: Admin xác nhận xóa → hệ thống kiểm tra ràng buộc (sản phẩm có trong đơn hàng chưa) → thực hiện xóa hoặc báo lỗi. 4. Giao diện cập nhật lại danh sách sau mỗi thao tác.
Kết quả	Dữ liệu sản phẩm trong hệ thống được cập nhật theo thao tác của Admin.

2.4.8. Quản lý đơn hàng (Admin)

Tên Use Case	Quản lý đơn hàng
Tác nhân	Admin
Mô tả	Admin theo dõi danh sách đơn hàng của người dùng, xem chi tiết từng đơn và cập nhật trạng thái xử lý.
Điều kiện trước	Admin đã đăng nhập; hệ thống có ít nhất một đơn hàng được tạo.
Luồng xử lý chính	1. Admin truy cập màn hình Quản lý đơn hàng. 2. Hệ thống truy vấn bảng Orders, hiển thị danh sách theo thứ tự mới nhất trước. 3. Admin nhấn vào đơn hàng để xem chi tiết (sản phẩm, số lượng, tổng tiền, địa chỉ...). 4. Admin cập nhật trạng thái đơn hàng (Đang xử lý / Đang giao / Đã giao / Đã hủy). 5. Hệ thống lưu trạng thái mới vào bảng Orders.
Kết quả	Trạng thái đơn hàng được cập nhật; User có thể thấy trạng thái mới trong lịch sử mua hàng.

2.5. Thiết kế cơ sở dữ liệu

Sau khi hoàn thành phân tích chức năng, bước tiếp theo là thiết kế cơ sở dữ liệu — xác định cách tổ chức và lưu trữ thông tin phục vụ cho toàn bộ hệ thống. Cơ sở dữ liệu được xây

dựng theo mô hình quan hệ (Relational Model) và triển khai trên SQLite — hệ quản trị CSDL nhúng sẵn trong Android, phù hợp với quy mô ứng dụng cục bộ của đề tài.

Hệ thống sử dụng 7 bảng dữ liệu: User, Admin, Category, Product, Cart, CartItem, Orders và OrderDetail. Các bảng liên kết với nhau qua khóa chính (Primary Key) và khóa ngoại (Foreign Key), đảm bảo tính toàn vẹn dữ liệu và hạn chế trùng lặp.

2.5.1. Phân tích dữ liệu cần lưu trữ

Từ các chức năng đã phân tích, hệ thống cần lưu trữ các nhóm thông tin sau:

- Thông tin tài khoản người dùng: họ tên, email (định danh duy nhất), mật khẩu, số điện thoại, địa chỉ — phục vụ đăng nhập và quản lý đơn hàng.
- Thông tin quản trị viên: tách riêng khỏi bảng User để phân biệt rõ vai trò và đảm bảo kiểm soát quyền truy cập quản trị.
- Danh mục sản phẩm: tên danh mục, mô tả — dùng để phân loại và lọc sản phẩm theo nhóm.
- Sản phẩm: tên, giá, số lượng tồn kho, hình ảnh, mô tả, thuộc danh mục nào — thông tin cốt lõi hiển thị cho người dùng.
- Giỏ hàng: thông tin giỏ thuộc user nào (Cart) và từng sản phẩm cụ thể trong giỏ (CartItem).
- Đơn hàng: thông tin tổng quát mỗi lần mua (Orders) và danh sách sản phẩm trong đơn (OrderDetail), bao gồm giá tại thời điểm đặt hàng.

2.5.2. Bảng User (Người dùng)

Lưu thông tin tài khoản khách hàng. Trường email được ràng buộc UNIQUE, đảm bảo mỗi địa chỉ email chỉ được đăng ký một lần. Trường diaChi được dùng làm địa chỉ nhận hàng mặc định khi đặt hàng.

```
CREATE TABLE User (  
    userID      INTEGER PRIMARY KEY AUTOINCREMENT,  
    hoTen       TEXT      NOT NULL,  
    email       TEXT      UNIQUE NOT NULL,  
    matKhau     TEXT      NOT NULL,  
    soDienThoai TEXT      NOT NULL,  
    diaChi      TEXT      NOT NULL  
);
```

2.5.3. Bảng Category (Danh mục sản phẩm)

Lưu danh mục sản phẩm (Điện thoại, Laptop, Tai nghe, Phụ kiện...). Bảng này tách riêng giúp dễ mở rộng danh mục mà không ảnh hưởng đến cấu trúc bảng Product. Mỗi sản phẩm tham chiếu đến một danh mục qua categoryID.

```
CREATE TABLE Category (
    categoryID INTEGER PRIMARY KEY AUTOINCREMENT,
    tenDanhMuc TEXT NOT NULL,
    moTa TEXT
);
```

2.5.4. Bảng Product (Sản phẩm)

Lưu thông tin chi tiết sản phẩm bán trên ứng dụng. Trường hìnhAnh lưu tên file ảnh (đặt trong thư mục assets/images của dự án). Trường soLuong phản ánh tồn kho và có thể được Admin cập nhật.

```
CREATE TABLE Product (
    productID INTEGER PRIMARY KEY AUTOINCREMENT,
    categoryID INTEGER NOT NULL,
    tenSP TEXT NOT NULL,
    gia REAL NOT NULL,
    soLuong INTEGER NOT NULL,
    hìnhAnh TEXT,
    moTa TEXT,
    FOREIGN KEY (categoryID) REFERENCES Category(categoryID)
);
```

2.5.5. Bảng Cart và CartItem (Giỏ hàng)

Giỏ hàng được thiết kế thành hai bảng để phân tách thông tin tổng quan (Cart) và từng dòng sản phẩm cụ thể (CartItem). Cách thiết kế này cho phép một giỏ hàng chứa nhiều sản phẩm khác nhau và dễ dàng cập nhật số lượng từng mặt hàng độc lập.

Bảng Cart:

```
CREATE TABLE Cart(
    cartID TEXT PRIMARY KEY,
    userID TEXT NOT NULL,
    tenUser TEXT NOT NULL,
    tongTien REAL,
    FOREIGN KEY(userID) REFERENCES User(userID)
);
```

Bảng CartItem:

```
CREATE TABLE CartItem(
    cartItemID TEXT PRIMARY KEY,
    cartID TEXT NOT NULL,
    productID TEXT NOT NULL,
    tenSP TEXT NOT NULL,
```

```
soLuong INTEGER NOT NULL,  
FOREIGN KEY(cartID) REFERENCES Cart(cartID),  
FOREIGN KEY(productID) REFERENCES Product(productID)  
);
```

2.5.6. Bảng Orders và OrderDetail (Đơn hàng)

Tương tự giỏ hàng, đơn hàng được tách thành hai bảng. Orders lưu thông tin tổng quát mỗi giao dịch; OrderDetail lưu từng dòng sản phẩm với đơn giá tại thời điểm đặt hàng — điều này quan trọng vì giá sản phẩm có thể thay đổi sau đó, nhưng lịch sử đơn hàng cần phản ánh giá thực tế khi mua.

Bảng Orders:

```
CREATE TABLE Orders(  
orderID TEXT PRIMARY KEY,  
userID TEXT NOT NULL,  
tenUser TEXT NOT NULL,  
ngayDat TEXT NOT NULL,  
trangThai TEXT NOT NULL,  
tongTien REAL NOT NULL,  
diaChiNhan TEXT NOT NULL,  
phuongThucThanhToan TEXT NOT NULL,  
FOREIGN KEY(userID) REFERENCES User(userID)  
);
```

Bảng OrderDetail:

```
CREATE TABLE OrderDetail(  
detailID TEXT PRIMARY KEY,  
orderID TEXT NOT NULL,  
productID TEXT NOT NULL,  
tenSP TEXT NOT NULL,  
soLuong INTEGER NOT NULL,  
donGia REAL NOT NULL,  
FOREIGN KEY(orderID) REFERENCES Orders(orderID),  
FOREIGN KEY(productID) REFERENCES Product(productID)  
);
```


2.5.7. Bảng Admin (Quản trị viên)

Lưu thông tin tài khoản quản trị viên, tách biệt hoàn toàn với bảng User. Tài khoản Admin không được tạo qua giao diện đăng ký mà được thêm trực tiếp vào CSDL. Trường chucVu ghi nhận vai trò cụ thể của từng Admin trong hệ thống

```
CREATE TABLE Admin (  
    adminID    INTEGER PRIMARY KEY AUTOINCREMENT,  
    hoTen      TEXT    NOT NULL,  
    email      TEXT    UNIQUE NOT NULL,  
    matKhau    TEXT    NOT NULL,  
    soDienThoai TEXT    NOT NULL,  
    chucVu     TEXT    NOT NULL  
);
```

2.6. Mô hình quan hệ cơ sở dữ liệu (ERD)

Mô hình ERD (Entity Relationship Diagram) thể hiện trực quan cấu trúc các bảng dữ liệu trong hệ thống bán hàng online và mối quan hệ giữa chúng. Thông qua ERD, có thể dễ dàng hiểu được cách dữ liệu được tổ chức, liên kết và trao đổi trong toàn bộ hệ thống.

Bảng dưới đây mô tả các bảng chính, khóa chính (Primary Key), khóa ngoại (Foreign Key) và ý nghĩa liên kết:

Bảng 2.3. Danh sách khóa chính và khóa ngoại trong cơ sở dữ liệu

Bảng dữ liệu	Khóa chính (PK)	Khóa ngoại (FK)	Ý nghĩa liên kết
User	userID	—	Lưu thông tin người dùng; là trung tâm của Cart và Orders
Admin	adminID	—	Tài khoản quản trị hệ thống, hoạt động độc lập với User
Category	categoryID	—	Danh mục sản phẩm trong hệ thống
Product	productID	categoryID Category	→ Mỗi sản phẩm thuộc một danh mục cụ thể
Cart	cartID	userID → User	Mỗi giỏ hàng gắn với một người dùng
CartItem	cartItemID	cartID → Cart, productID → Product	Chi tiết sản phẩm trong giỏ hàng
Orders	orderId	userID → User	Mỗi đơn hàng được tạo bởi một người dùng
OrderDetail	detailID	orderId → Orders, productID → Product	Chi tiết từng sản phẩm trong đơn hàng

2.6.1. Các mối quan hệ chính

Hệ thống được xây dựng dựa trên các mối quan hệ một-nhiều (1–N), phản ánh đúng quy trình hoạt động của một hệ thống bán hàng online:

Category (1) — (N) Product: Một danh mục có thể chứa nhiều sản phẩm, nhưng mỗi sản phẩm chỉ thuộc một danh mục duy nhất.

User (1) — (N) Cart: Một người dùng có thể có nhiều giỏ hàng trong các lần sử dụng khác nhau của hệ thống.

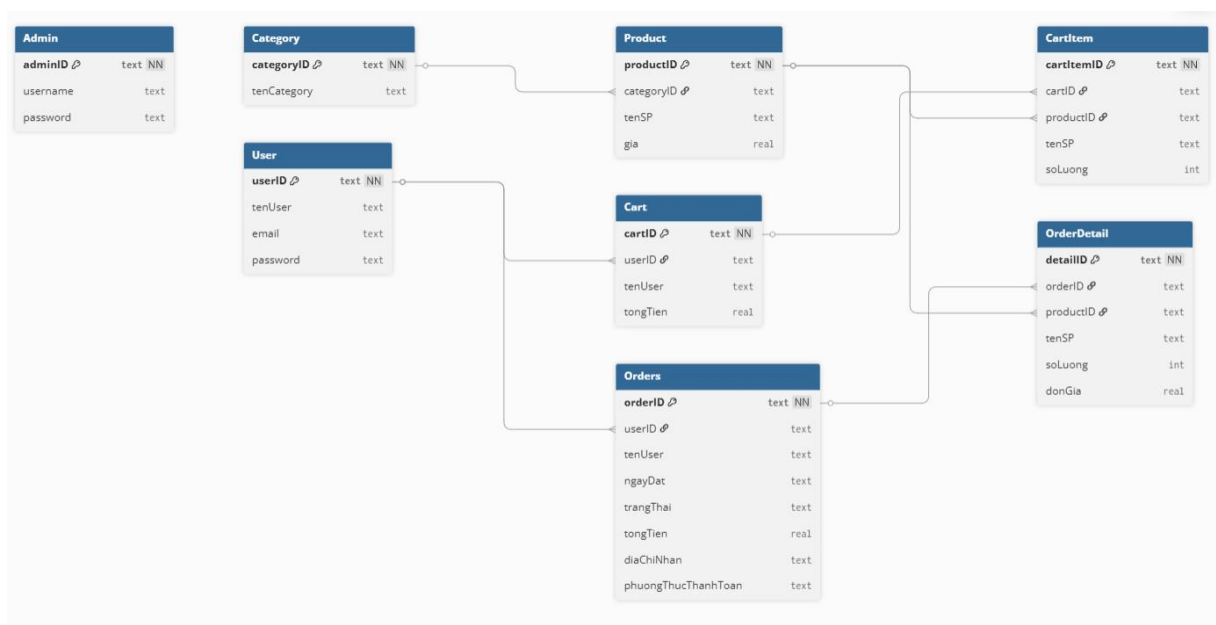
Cart (1) — (N) CartItem: Một giỏ hàng bao gồm nhiều sản phẩm được thêm vào.

Product (1) — (N) CartItem: Một sản phẩm có thể xuất hiện trong nhiều giỏ hàng của các người dùng khác nhau.

User (1) — (N) Orders: Một người dùng có thể tạo nhiều đơn hàng trong quá trình mua sắm.

Orders (1) — (N) OrderDetail: Một đơn hàng có thể chứa nhiều sản phẩm khác nhau.

Product (1) — (N) OrderDetail: Một sản phẩm có thể xuất hiện trong nhiều đơn hàng khác nhau. Giá trị đơn giá được lưu theo thời điểm đặt hàng để đảm bảo tính lịch sử dữ liệu.



Hình 2.5. Sơ đồ ERD hệ thống bán hàng online trên Android

Sơ đồ ERD trên mô tả cấu trúc cơ sở dữ liệu của hệ thống bán hàng online và mối quan hệ giữa các bảng trong hệ thống. Các bảng được thiết kế nhằm đảm bảo dữ liệu

được tổ chức chặt chẽ, hạn chế trùng lặp và phản ánh đúng quy trình nghiệp vụ mua hàng.

Trong hệ thống, bảng User đóng vai trò trung tâm, lưu trữ thông tin người dùng và liên kết với các bảng Cart và Orders thông qua khóa ngoại userID. Mỗi người dùng có thể sở hữu một hoặc nhiều giỏ hàng cũng như tạo nhiều đơn hàng trong quá trình sử dụng hệ thống.

Bảng Cart được sử dụng để lưu thông tin giỏ hàng tạm thời của người dùng, trong khi bảng CartItem chứa danh sách các sản phẩm có trong giỏ hàng. Mỗi CartItem sẽ liên kết với một Cart cụ thể và một Product tương ứng, thể hiện mối quan hệ giữa người dùng và sản phẩm đã chọn.

Bảng Orders lưu thông tin đơn hàng đã được xác nhận, mỗi đơn hàng thuộc về một người dùng cụ thể. Chi tiết từng đơn hàng được lưu trong bảng OrderDetail, trong đó mỗi dòng thể hiện một sản phẩm cụ thể thuộc đơn hàng đó.

Bảng Product lưu thông tin sản phẩm và được liên kết với bảng Category để phân loại sản phẩm theo danh mục. Đồng thời, Product cũng liên kết với CartItem và OrderDetail nhằm thể hiện sự xuất hiện của sản phẩm trong giỏ hàng và đơn hàng.

Bảng Admin được thiết kế độc lập với các bảng nghiệp vụ khác, dùng để quản lý và điều hành hệ thống. Admin không tham gia trực tiếp vào quá trình mua hàng nên không có quan hệ khóa ngoại với các bảng còn lại.

Chung quy lại, mô hình ERD đảm bảo các mối quan hệ một-nhiều (1-N) giữa các thực thể chính, giúp hệ thống hoạt động ổn định, dễ mở rộng và phù hợp với mô hình ứng dụng bán hàng trên nền tảng Android.

2.7. Thiết kế Sequence Diagram (Sơ đồ tuần tự)

Sequence Diagram mô tả trình tự trao đổi thông tin giữa các thành phần của hệ thống khi thực hiện một chức năng cụ thể. Đây là công cụ giúp làm rõ luồng xử lý bên trong ứng dụng trước khi lập trình, đặc biệt hữu ích để xác định trách nhiệm của từng Activity và cách chúng tương tác với CSDL.

Ba luồng quan trọng nhất được đặc tả dưới đây.

2.7.1. Luồng đăng nhập và phân quyền

Chức năng đăng nhập là điểm khởi đầu của toàn bộ hệ thống, cho phép xác thực người dùng và phân quyền truy cập giữa User và Admin. Sau khi nhận thông tin từ người

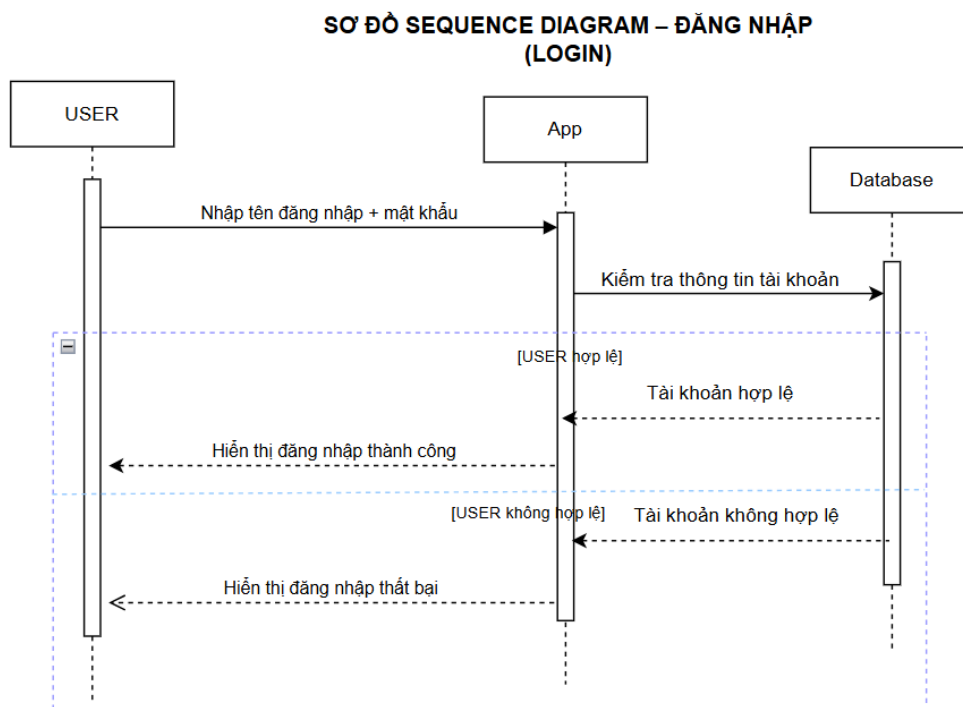
dùng, hệ thống tiến hành kiểm tra trong cơ sở dữ liệu và điều hướng đến giao diện phù hợp.

Bảng 2.4. Trình tự xử lý luồng đăng nhập và phân quyền

Bước	Đối tượng	Hành động / Nội dung xử lý
1	User	Nhập email và mật khẩu, nhấn “Đăng nhập”
2	LoginActivity	Nhận sự kiện click, lấy dữ liệu từ EditText
3	LoginActivity SQLite	→ Truy vấn bảng User: <code>SELECT * FROM User WHERE email=? AND password=?</code>
4	SQLite LoginActivity	→ Trả về kết quả truy vấn
5	LoginActivity	Kiểm tra kết quả (ALT)
6	ALT – User hợp lệ	Lưu userID vào SharedPreferences → chuyển đến HomeActivity
7	ALT – Admin hợp lệ	Chuyển đến AdminDashboardActivity
8	ALT – Không hợp lệ	Hiển thị thông báo “Sai tài khoản hoặc mật khẩu”

Hệ thống sử dụng cơ chế phân quyền đơn giản giữa User và Admin.

Việc điều hướng giao diện được thực hiện sau khi xác thực thành công



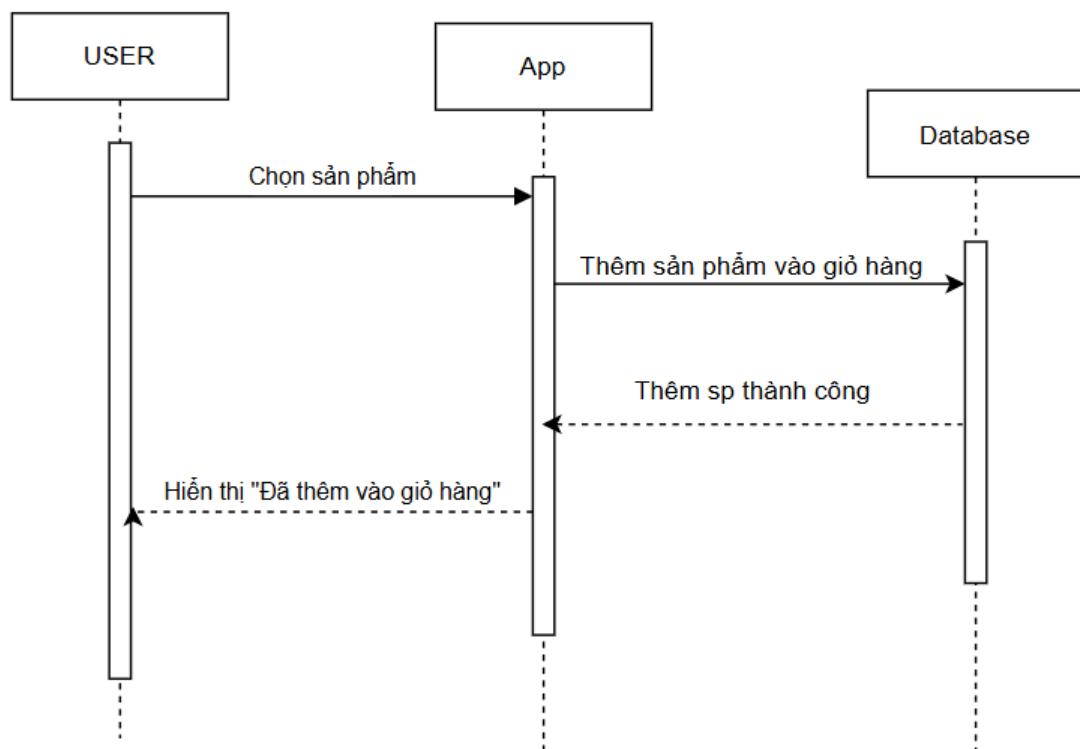
(Hình 2.6. Sequence Diagram luồng đăng nhập và phân quyền)

2.7.2. Luồng thêm vào giỏ hàng

Bảng 2.5. Trình tự xử lý thêm giỏ hàng

Bước	Đối tượng	Hành động
1	User	Chọn sản phẩm
2	ProductDetailActivity	Lấy productID
3	SQLite	Kiểm tra CartItem
4	ALT	Thêm mới hoặc cập nhật số lượng
5	SQLite	Cập nhật Cart

SƠ ĐỒ SEQUENCE DIAGRAM – THÊM VÀO GIỎ HÀNG (CART)



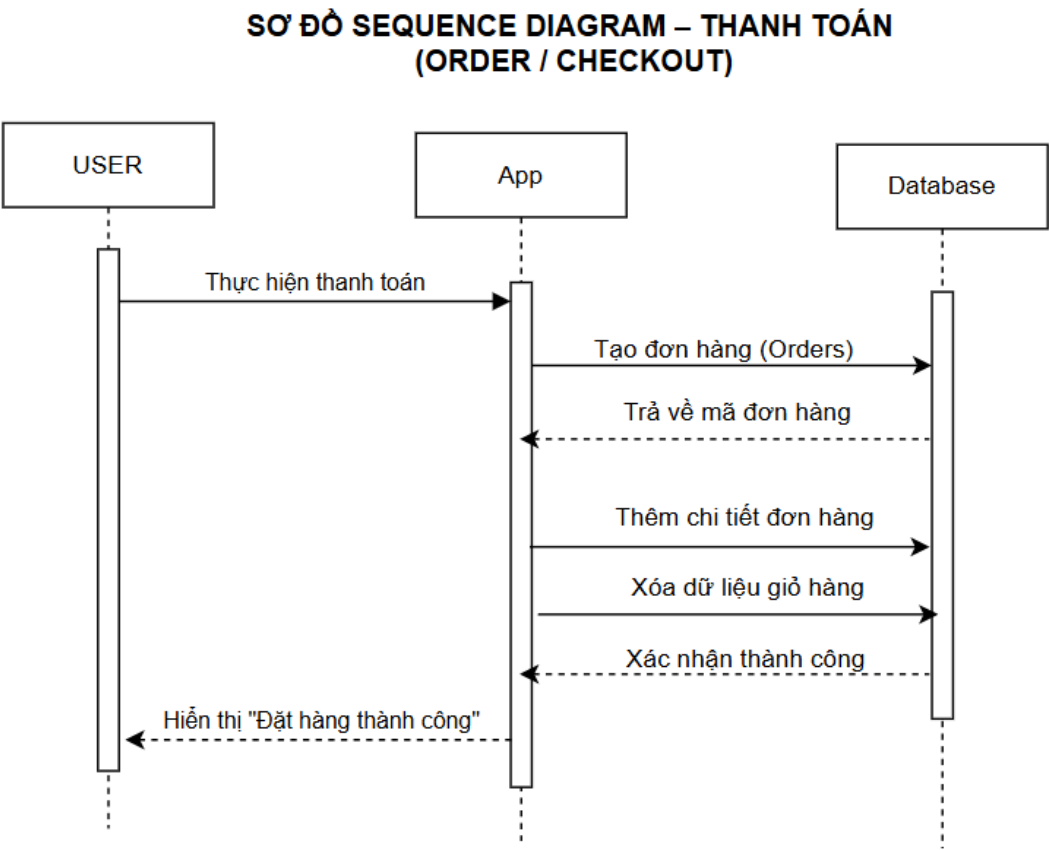
Hình 2.7. Sequence Diagram thêm vào giỏ hàng

Luồng này mô tả quá trình người dùng thêm sản phẩm vào giỏ hàng. Hệ thống kiểm tra sản phẩm đã tồn tại trong giỏ hay chưa, sau đó thực hiện thêm mới hoặc cập nhật số lượng tương ứng. Cuối cùng dữ liệu giỏ hàng được cập nhật trong cơ sở dữ liệu.

2.7.3. *Luồng thanh toán (Checkout)*

Bảng 2.6. Trình tự xử lý thanh toán

Bước	Đối tượng	Hành động
1	User	Nhấn thanh toán
2	CheckoutActivity	Xử lý đơn hàng
3	SQLite	Tạo Orders
4	SQLite	Tạo OrderDetail
5	SQLite	Xóa CartItem
6	App	Hiển thị thông báo thành công



Hình 2.8. Sequence Diagram thanh toán

Luồng thanh toán mô tả toàn bộ quá trình tạo đơn hàng từ giỏ hàng của người dùng. Hệ thống tạo đơn hàng mới, lưu chi tiết sản phẩm và xóa dữ liệu giỏ hàng sau khi thanh toán thành công, đảm bảo tính nhất quán dữ liệu.

2.8. Thiết kế giao diện hệ thống (Wireframe)

Thiết kế wireframe là bước lập kế hoạch giao diện trước khi viết code. Ở giai đoạn này, mỗi màn hình được mô tả về bố cục, vị trí thành phần và luồng điều hướng — không đi vào màu sắc hay chi tiết đồ họa. Kết quả của bước này là cơ sở trực tiếp để xây dựng các file layout XML trong Android Studio.

Giao diện ứng dụng được chia thành hai nhóm: giao diện dành cho User (luồng mua sắm) và giao diện dành cho Admin (khu vực quản trị).

2.8.1. Giao diện dành cho User

a) Màn hình Login (Đăng nhập)

Màn hình đầu tiên người dùng gặp khi mở ứng dụng. Bố cục đơn giản, đặt trung tâm: logo/tên ứng dụng, ô nhập email, ô nhập mật khẩu, nút đăng nhập và liên kết chuyển sang màn hình đăng ký. Không hiển thị thông tin không cần thiết để tránh gây rối.

Điều hướng sau đăng nhập: nếu là User → HomeActivity; nếu là Admin → AdminDashboardActivity.

b) Màn hình Register (Đăng ký)

Cho phép tạo tài khoản mới. Gồm các ô nhập: họ tên, email, mật khẩu, số điện thoại, địa chỉ và nút xác nhận đăng ký. Toàn bộ bố cục đặt trong ScrollView để phù hợp với màn hình nhỏ. Các ô bắt buộc có thông báo lỗi nếu bỏ trống.

c) Màn hình Home (Trang chủ)

Giao diện chính sau đăng nhập. Bố cục từ trên xuống gồm: thanh tiêu đề ứng dụng + icon giỏ hàng góc phải; thanh tìm kiếm; danh sách danh mục ngang (RecyclerView horizontal); lưới sản phẩm nổi bật (RecyclerView dạng Grid 2 cột); thanh điều hướng phía dưới (Home / Sản phẩm / Giỏ hàng / Hồ sơ).

d) Màn hình Product List (Danh sách sản phẩm)

Hiển thị sản phẩm dạng lưới 2 cột bằng RecyclerView + GridLayoutManager. Mỗi item gồm: hình ảnh sản phẩm (ImageView, tỉ lệ vuông), tên sản phẩm (TextView, tối đa 2 dòng), giá bán (TextView, in đậm, màu nhấn). Header có nút lọc theo danh mục.

e) Màn hình Product Detail (Chi tiết sản phẩm)

Bố cục dọc trong ScrollView: hình ảnh lớn sản phẩm (chiếm 40% chiều cao màn hình), tên sản phẩm, giá bán nổi bật, mô tả chi tiết, thông số kỹ thuật (nếu có). Nút "Thêm vào giỏ hàng" cố định ở phía dưới màn hình (không cuộn theo nội dung).

f) Màn hình Cart (Giỏ hàng)

Danh sách sản phẩm trong giỏ bằng RecyclerView. Mỗi item gồm: hình ảnh nhỏ, tên, đơn giá, bộ tăng/giảm số lượng và nút xóa. Phía dưới hiển thị tổng tiền (tự động cập nhật khi thay đổi số lượng) và nút "Đặt hàng".

g) Màn hình Checkout (Xác nhận đặt hàng)

Hiển thị tóm tắt đơn hàng: danh sách sản phẩm đã chọn, thông tin người nhận (địa chỉ, số điện thoại — tự điền từ hồ sơ), tổng tiền, lựa chọn phương thức thanh toán giả lập (Tiền mặt / Chuyển khoản), nút xác nhận. Đây là màn hình không thể thao tác khi giỏ hàng trống.

h) Màn hình Order History (Lịch sử đơn hàng)

Danh sách đơn hàng sắp theo thứ tự mới nhất trước. Mỗi item hiển thị: mã đơn hàng, ngày đặt, tổng tiền và trạng thái (chip màu phân biệt: Đang xử lý / Đang giao / Đã giao / Đã hủy). Nhấn vào một đơn để xem chi tiết sản phẩm bên trong.

i) Màn hình Profile (Hồ sơ cá nhân)

Hiển thị thông tin tài khoản: họ tên, email, số điện thoại, địa chỉ. Người dùng có thể chỉnh sửa các trường (trừ email là định danh). Có nút đăng xuất ở cuối trang.

2.8.2. Giao diện dành cho Admin

a) Dashboard (Tổng quan)

Màn hình đầu tiên Admin thấy sau khi đăng nhập. Hiển thị các thẻ thống kê tổng quan: tổng sản phẩm, tổng đơn hàng, số lượng người dùng và doanh thu giả lập. Có menu điều hướng đến các chức năng quản lý.

b) Product Management (Quản lý sản phẩm)

Danh sách toàn bộ sản phẩm hiện có, mỗi dòng hiển thị tên, danh mục, giá và hai nút hành động [Sửa] / [Xóa]. Nút [+ Thêm sản phẩm] ở góc dưới phải (FAB — Floating Action Button) để chuyển đến màn hình thêm mới.

c) Add/Edit Product (Thêm/Sửa sản phẩm)

Màn hình dùng chung cho thêm mới và chỉnh sửa sản phẩm. Khi sửa, các trường được điền sẵn giá trị hiện tại. Gồm: ô tên sản phẩm, spinner chọn danh mục, ô giá, ô số lượng, tên file ảnh, ô mô tả và nút lưu.

d) Order Management (Quản lý đơn hàng)

Danh sách đơn hàng với thông tin: mã đơn, tên người mua, tổng tiền, trạng thái hiện tại. Admin nhấn vào một đơn để xem chi tiết và cập nhật trạng thái qua dropdown (Đang xử lý / Đang giao / Đã giao / Đã hủy).

e) User Management (Quản lý người dùng)

Hiển thị danh sách tài khoản người dùng: ID, họ tên, email, số điện thoại. Chức năng ở giai đoạn này chủ yếu là xem; không cho phép Admin tạo tài khoản User hay xóa tài khoản đang có đơn hàng.

f) Report (Báo cáo thống kê)

Hiển thị các số liệu thống kê cơ bản: tổng số đơn hàng theo trạng thái, top sản phẩm bán chạy (tính từ OrderDetail), doanh thu giả lập theo tháng. Dữ liệu được truy vấn trực tiếp từ SQLite và hiển thị dạng bảng hoặc biểu đồ đơn giản.

CHƯƠNG 3. XÂY DỰNG VÀ KẾT QUẢ THỰC NGHIỆM

Sau khi đã hoàn tất các bước phân tích yêu cầu và thiết kế hệ thống ở Chương 2 bao gồm xác định Use Case, xây dựng Class Diagram, thiết kế mô hình dữ liệu mức quan niệm (ERD), mô tả Sequence Diagram cho ba luồng nghiệp vụ chính và phác thảo wireframe cho từng màn hình Chương 3 trình bày quá trình hiện thực hoá các thiết kế đó thành một ứng dụng Android chạy được trên thực tế, đồng thời phân tích chi tiết kết quả giao diện và dữ liệu mà ứng dụng SaleApp đã đạt được sau khi hoàn thành phân lập trình.

Trong quá trình lập trình, do giới hạn về thời gian thực hiện bài tập lớn, một số quyết định kỹ thuật đã được điều chỉnh so với bản thiết kế ban đầu nhằm đảm bảo tiến độ hoàn thành toàn bộ luồng giao diện và nghiệp vụ chính. Những điều chỉnh này, đặc biệt là ở phần tổ chức dữ liệu, sẽ được trình bày minh bạch và lý giải cụ thể ngay trong mục tương ứng (mục 3.2), thay vì để đến phần đánh giá mới đề cập, nhằm giúp người đọc theo dõi báo cáo một cách nhất quán và trung thực với những gì đã thực sự xây dựng được.

3.1. Môi trường phát triển

Phần này trình bày môi trường thực tế mà em sử dụng trong suốt quá trình xây dựng ứng dụng SaleApp, bao gồm cấu hình máy tính dùng để viết code, biên dịch và chạy giả lập, cùng các phần mềm, công cụ hỗ trợ trong từng giai đoạn của quá trình phát triển. Việc trình bày rõ môi trường phát triển không chỉ nhằm mục đích mô tả, mà còn giúp làm rõ lý do một số lựa chọn kỹ thuật được đưa ra ở các mục tiếp theo, chẳng hạn lý do lựa chọn các phiên bản SDK tối thiểu/tối đa, hay lý do kiểm thử song song trên cả giả lập và thiết bị thật.

3.1.1. Phần mềm sử dụng

- Android Studio: phiên bản Android Studio Ladybug, dùng làm IDE chính cho toàn bộ quá trình thiết kế giao diện, viết code Kotlin và kiểm thử ứng dụng.
- Ngôn ngữ lập trình: Kotlin, kết hợp với XML cho phần giao diện.
- SDK: target SDK 34 (Android 14), minimum SDK 24 (Android 7.0) để vẫn đảm bảo ứng dụng chạy được trên các thiết bị đời cũ hơn.
- Thiết bị thử nghiệm: chủ yếu dùng máy giả lập Pixel 6 (Android 14) tích hợp trong Android Studio; ngoài ra có cài thử trên điện thoại thật (Xiaomi, Android 13) để kiểm tra độ ổn định trên thiết bị thực tế.

Một lựa chọn kỹ thuật quan trọng được áp dụng nhất quán xuyên suốt dự án là kích hoạt tính năng View Binding ngay từ đầu (`viewBinding { enable = true }` trong file `build.gradle` cấp module). Nhờ vậy, toàn bộ 16 Activity và 6 Adapter trong ứng dụng (ProductAdapter,

CategoryAdapter, CartAdapter, AdminOrderAdapter, AdminProductAdapter, AdminUserAdapter) đều truy cập các thành phần giao diện thông qua một lớp Binding sinh tự động (ví dụ ActivityHomeBinding, ItemProductBinding) thay vì gọi findViewById() thủ công. Cách làm này giúp loại bỏ hoàn toàn lỗi ép kiểu sai hoặc NullPointerException do gõ sai id view, một lỗi khá thường gặp khi project có số lượng màn hình lớn dần theo thời gian.

Việc kiểm thử song song trên cả giả lập Pixel 6 và điện thoại Xiaomi thật giúp em phát hiện được một số khác biệt nhỏ về khoảng cách (margin) và cỡ chữ giữa hai loại màn hình, từ đó điều chỉnh lại một vài giá trị dp/sp trong file layout cho cân đối hơn, đặc biệt ở các màn hình có nhiều thành phần xếp chồng như Product Detail và Checkout, điều mà nếu chỉ kiểm thử trên giả lập sẽ khó nhận ra ngay.

3.2. Xây dựng tầng dữ liệu trung tâm của ứng dụng (AppDataStore)

Ở Chương 2, mục 2.5 và 2.6, em đã thiết kế một mô hình cơ sở dữ liệu quan hệ tương đối đầy đủ gồm 8 thực thể (User, Admin, Category, Product, Cart, CartItem, Orders, OrderDetail) với các mối quan hệ một-nhiều giữa các bảng, và từ những phần dữ liệu đã được nhập trong SQLite để kết nối, đưa lên hệ thống chính rồi hiển thị đầy đủ các thông tin.

Tuy nhiên, trong quá trình triển khai thực tế, vì bài tập lớn có thời lượng thực hiện có hạn và trọng tâm đặt ra là hoàn thiện đầy đủ phần giao diện cùng các luồng tương tác chính của một ứng dụng bán hàng (điều hướng giữa các màn hình, xử lý sự kiện người dùng, hiển thị dữ liệu động), phần triển khai dữ liệu SQLite tách biệt không thêm vào hệ thống phần mềm, là phân tệp tài liệu tách biệt, không nối với các phần chương trình trong hệ thống.

3.2.1. Các lớp dữ liệu (data class) thực tế

Tương ứng với các thực thể đã thiết kế ở Chương 2, năm lớp dữ liệu sau được hiện thực bằng data class của Kotlin một loại class đặc biệt tự động sinh các phương thức equals(), hashCode(), toString() và đặc biệt là copy(), giúp việc so sánh và sao chép đối tượng trở nên ngắn gọn hơn rất nhiều so với việc tự viết tay trong Java.

- Category(id, name): tương ứng đúng thực thể Category ở Chương 2, lưu danh sách 5 danh mục cố định (bao gồm mục “Tất cả” dùng riêng cho bộ lọc, không phải một danh mục sản phẩm thật).
- Product(id, name, price, category, image, rating, reviews, stock, desc, quantity = 1): gần như đầy đủ các trường đã thiết kế cho thực thể Product. Điểm khác biệt là trường quantity (mặc định = 1) được gộp thêm ngay trong Product để tái sử dụng cho cả hai

vai trò vừa là sản phẩm trong danh mục (quantity không có ý nghĩa) vừa là một dòng trong giỏ hàng (quantity thể hiện số lượng khách chọn mua).

- User(id, name, email): được rút gọn nhiều so với thiết kế ban đầu (vốn có thêm mật khẩu, số điện thoại, địa chỉ); ở bản hiện tại, User chỉ đóng vai trò hiển thị danh sách khách hàng phía Admin, chưa gắn với cơ chế đăng nhập thật.
- Order(id, customerName, status, totalAmount, items: List<OrderItem>): gộp luôn vai trò của cả hai thực thể Orders và OrderDetail đã thiết kế ở Chương 2 vào trong một đối tượng duy nhất, bằng cách nhúng trực tiếp danh sách items thay vì tách thành một bảng OrderDetail riêng có khoá ngoại orderID.
- OrderItem(productId, productName, quantity, unitPrice): tương đương các trường quan trọng của OrderDetail. Đáng chú ý, unitPrice được lưu lại tại thời điểm đặt hàng (sao chép từ Product.price khi tạo đơn) thay vì tham chiếu lại Product.price hiện tại đúng với nguyên tắc “đảm bảo tính lịch sử dữ liệu” đã đặt ra khi thiết kế quan hệ Product – OrderDetail ở mục 2.6.1, dù ở đây được thực hiện bằng cách sao chép giá trị thủ công thay vì qua ràng buộc của CSDL.

3.2.2. Các nhóm hàm xử lý dữ liệu và vai trò tương đương trong mô hình CSDL truyền thống

Vì không sử dụng SQL, mọi phép toán truy vấn, thêm, sửa, xoá dữ liệu trong AppDataStore đều được viết lại bằng các hàm xử lý danh sách (List) thuần Kotlin. Bảng 3.1 tổng hợp các hàm chính cùng câu lệnh SQL tương đương về mặt ý nghĩa, giúp liên hệ trực tiếp với mô hình đã thiết kế ở Chương 2.

Bảng 3.1. Các hàm xử lý dữ liệu trong AppDataStore và câu lệnh SQL tương đương về ý nghĩa

Hàm trong AppDataStore	Vai trò	Câu lệnh SQL tương đương
findProductById(id)	Tìm một sản phẩm theo mã	SELECT * FROM Product WHERE productID = ?
nextProductId()	Sinh mã sản phẩm mới tăng dần	Tương đương cơ chế AUTOINCREMENT
addOrUpdateProduct(p)	Thêm sản phẩm mới hoặc cập nhật nếu đã tồn tại	INSERT ... hoặc UPDATE Product SET ... WHERE productID = ?

Hàm trong AppDataStore	Vai trò	Câu lệnh SQL tương đương
addToCart(product)	Thêm sản phẩm vào giỏ, tăng số lượng nếu đã có	INSERT INTO CartItem hoặc UPDATE CartItem SET quantity = quantity + 1
cartTotal()	Tính tổng tiền giỏ hàng	SELECT SUM(gia * soLuong) FROM CartItem
clearCart()	Xoá toàn bộ sản phẩm trong giỏ sau khi đặt hàng	DELETE FROM CartItem WHERE cartID = ?
placeOrder(name)	Tạo đơn hàng mới từ giỏ hàng hiện tại	INSERT INTO Orders ...; INSERT INTO OrderDetail ... (transaction)

Trong số các hàm trên, addToCart() thể hiện rõ nhất một chi tiết kỹ thuật quan trọng mà em đặc biệt lưu ý khi lập trình:

```
fun addToCart(product: Product) {
    val existing = cartItems.find { it.id == product.id }
    if (existing != null) {
        existing.quantity++
    } else {
        cartItems.add(product.copy(quantity = 1))
    }
}
```

Nếu sản phẩm đã tồn tại trong giỏ, hàm chỉ tăng quantity của bản ghi đó lên 1. Nhưng nếu là sản phẩm mới, thay vì thêm trực tiếp tham chiếu product hiện có (vốn đang được chia sẻ với danh sách productCatalog đang hiển thị ở Home và Product List), hàm gọi product.copy(quantity = 1) để tạo ra một đối tượng Product hoàn toàn độc lập về vùng nhớ. Đây là một chi tiết nhỏ nhưng ảnh hưởng lớn đến tính đúng đắn của dữ liệu: vì data class trong Kotlin là kiểu tham chiếu (reference type), nếu không gọi copy(), việc tăng/giảm số lượng trong giỏ hàng ở CartAdapter (vốn thay đổi trực tiếp product.quantity) sẽ vô tình làm thay đổi luôn số lượng hiển thị của chính sản phẩm đó trong danh mục sản phẩm gốc một lỗi rất khó phát hiện nếu không kiểm thử kỹ. Việc tách bản sao độc lập ngay từ khi thêm vào giỏ giúp tránh hoàn toàn lỗi này.

Tương tự, việc sinh mã đơn hàng mới (orderCounter, khởi tạo bằng 8926 và tăng dần sau mỗi lần đặt hàng để tạo ra các mã như DH-8926, DH-8927,...) là cách mô phỏng thủ công

cơ chế AUTOINCREMENT của SQLite ngay trong bộ nhớ. Hàm placeOrder() sau khi tạo Order mới sẽ chèn đơn hàng vào đầu danh sách bằng orders.add(0, newOrder) để đơn mới nhất luôn hiển thị trước đúng theo yêu cầu sắp xếp “mới nhất trước” đã đặt ra khi thiết kế màn hình Order History ở mục 2.8.1(h) sau đó gọi clearCart() để dọn sạch giỏ hàng, hoàn tất một giao dịch.

3.2.4. Dữ liệu mẫu thực tế

Khác với kế hoạch ban đầu là chèn khoảng 40 bản ghi mẫu thông qua một hàm insertSampleData() được gọi từ onCreate() của SQLite, bản triển khai thực tế khai báo trực tiếp một bộ dữ liệu mẫu nhỏ hơn ngay trong phần khởi tạo của AppDataStore, đủ để minh họa và kiểm thử toàn bộ các luồng nghiệp vụ chính (xem sản phẩm, thêm giỏ hàng, đặt hàng, quản trị) mà không cần một khối dữ liệu lớn gây khó theo dõi khi demo. Cụ thể, dữ liệu mẫu gồm 4 sản phẩm, mỗi sản phẩm thuộc một danh mục khác nhau, đủ đại diện cho cả 4 danh mục sản phẩm thật của hệ thống (Điện thoại, Laptop, Âm thanh, Phụ kiện).

Bảng 3.2. Dữ liệu mẫu sản phẩm thực tế được khai báo trong AppDataStore

id	Tên sản phẩm	Danh mục	Giá (đ)	Rating	Reviews	Stock
1	iPhone 15 Pro Max 256GB	Điện thoại	29.990.000	4.8	142	45
2	MacBook Air M3 13-inch	Laptop	27.490.000	4.9	88	20
3	Sony WH-1000XM5	Âm thanh	6.490.000	4.7	310	15
4	Chuột Logitech MX Master 3S	Phụ kiện	2.450.000	4.8	215	30

Ngoài ra, AppDataStore còn khai báo sẵn 3 tài khoản người dùng mẫu (dùng cho màn hình quản lý khách hàng phía Admin) và 2 đơn hàng mẫu một đơn ở trạng thái “Đã giao” (mã DH-8924, của khách Nguyễn Văn A) và một đơn ở trạng thái “Chờ duyệt” (mã DH-8925, của khách Lê Thị B) nhằm có sẵn dữ liệu minh họa ngay khi mở ứng dụng lần đầu, không cần phải tự tạo đơn hàng mới rồi mới có dữ liệu để xem ở các màn hình Order History, Admin Dashboard và Admin Orders.

3.3. Triển khai giao diện ứng dụng (kết quả thực tế)

Phần này trình bày kết quả giao diện thực tế của ứng dụng SaleApp sau khi đã hoàn thành lập trình. Khác với mục 2.8, vốn chỉ là bản phác thảo wireframe ở giai đoạn thiết kế mô tả bố cục và vị trí các thành phần, chưa có màu sắc, hiệu ứng hay dữ liệu thật các hình ảnh được chèn trong mục này (vị trí đã được đánh dấu rõ bằng khung trống kèm chú thích) là ảnh chụp

màn hình (screenshot) thật, được chụp trực tiếp từ ứng dụng đang chạy trên thiết bị/giả lập, có đầy đủ màu sắc, dữ liệu thật lấy từ AppDataStore và các hiệu ứng tương tác (đổi màu khi nhấn, hiển thị Toast, ẩn/hiện thành phần...).

Trước khi đi vào phân tích từng màn hình cụ thể, có một điểm chung về kỹ thuật trình bày giao diện cần nêu trước, vì nó được lặp lại nhất quán ở hầu hết các màn hình có danh sách dữ liệu (Home, Product List, Cart, Order History, và toàn bộ các màn hình quản lý phía Admin): toàn bộ các Activity này đều sử dụng RecyclerView kết hợp với một lớp Adapter riêng và View Binding, không dùng findViewById() ở bất kỳ đâu trong toàn bộ ứng dụng. Mỗi Adapter có một inner class ViewHolder giữ tham chiếu đến XxxBinding tương ứng (sinh tự động từ file layout item_XXX.xml), và hàm onBindViewHolder() chịu trách nhiệm đổ dữ liệu thật từ các data class (Product, Category, Order, User) vào các thành phần giao diện. Sự kiện click thường được gắn trực tiếp trên thành phần root (toàn bộ MaterialCardView của item), biến cả một “ô” trong danh sách thành một vùng bấm lớn duy nhất; một số item (như item_product.xml) còn có thêm một vùng bấm phụ độc lập (nút “+” thêm vào giỏ) để tách hai hành vi tương tác khác nhau xem chi tiết và thêm nhanh vào giỏ ngay trên cùng một item. Cách tổ chức nhất quán này giúp giao diện toàn ứng dụng có cùng một “ngôn ngữ tương tác”, dù được lập trình ở các màn hình khác nhau.

3.3.1. Giao diện Login

[Chèn ảnh chụp màn hình thực tế tại đây nền gradient xanh phía trên (drawable gradient_blue), card bo góc nổi lên chứa 2 ô nhập email/mật khẩu và nút “Đăng nhập”]

Hình 3.2. Giao diện màn hình Login của SaleApp

Giao diện Login được dựng trên ConstraintLayout, với một View nền phía trên cao 240dp sử dụng drawable gradient_blue (chuyển màu từ #4F46E5 sang #818CF8 theo góc 45 độ) tạo điểm nhấn thị giác ngay từ màn hình đầu tiên. Phần form đăng nhập được đặt trong một MaterialCardView bo góc 16dp, đổ bóng (cardElevation 8dp) và được kéo đề lên phần nền gradient nhờ thuộc tính layout_marginTop âm (-60dp) một kỹ thuật phổ biến trong thiết kế Material Design hiện đại để tạo hiệu ứng “card nổi” trên nền màu, giúp giao diện trông hiện đại hơn so với việc đặt các thành phần tách bạch theo từng khối vuông.

Về xử lý logic, LoginActivity.kt chỉ kiểm tra hai ô edtLoginEmail và edtLoginPassword không được để trống (sau khi .trim() khoảng trắng) trước khi cho phép đăng nhập; ứng dụng hiện tại chưa đối chiếu thông tin nhập vào với danh sách users trong AppDataStore, nghĩa là về bản chất bất kỳ cặp email/mật khẩu không rỗng nào cũng được coi là đăng nhập hợp lệ và được chuyển thẳng đến HomeActivity. Đây là điểm khác biệt rõ rệt so với luồng đăng nhập và phân quyền đã thiết kế ở mục 2.7.1 (vốn có truy vấn đối chiếu CSDL và rẽ nhánh User/Admin); ở phiên bản hiện tại, việc phân quyền chưa được hiện thực, mọi người dùng sau khi đăng nhập đều vào cùng một HomeActivity, và có thể tự do bấm vào nút “Chuyển sang giao diện Quản trị” ở màn Profile để vào khu vực Admin mà không qua bất kỳ bước xác thực quyền nào.

3.3.2. Giao diện Register

[Chèn ảnh chụp màn hình thực tế tại đây form đăng ký trong ScrollView với 4 ô nhập (họ tên, email, số điện thoại, mật khẩu) và nút “Đăng ký tài khoản”]

Hình 3.3. Giao diện màn hình Register của SaleApp

Toàn bộ form đăng ký được đặt trong một ScrollView bao ngoài LinearLayout dọc, đúng như đã thiết kế ở mục 2.8.1(b), nhằm đảm bảo 4 ô nhập liệu cùng nút đăng ký vẫn hiển thị đầy đủ và có thể cuộn được trên các màn hình có chiều cao nhỏ, hoặc khi bàn phím ảo được bật lên chiếm phần lớn diện tích màn hình.

RegisterActivity.kt kiểm tra cả 4 trường không được để trống trước khi cho phép đăng ký, hiển thị Toast “Vui lòng nhập đầy đủ thông tin” nếu thiếu. Tuy nhiên, một điểm cần lưu ý là khi đăng ký thành công, ứng dụng chỉ hiển thị Toast thông báo rồi điều hướng thẳng về LoginActivity, mà không thực hiện thêm tài khoản mới vào danh sách AppDataStore.users. Nói cách khác, màn hình đăng ký hiện tại hoàn chỉnh về giao diện và validate đầu vào, nhưng chưa có tác dụng thực sự lên dữ liệu hệ thống đây là một hạn chế sẽ được nêu lại ở mục 3.4.3.

3.3.3. Giao diện Home

[Chèn ảnh chụp màn hình thực tế tại đây Toolbar tùy biến với logo và 2 icon, thanh tìm kiếm, danh

mục cuộn ngang, lưới sản phẩm 2 cột, thanh điều hướng dưới]

Hình 3.4. Giao diện màn hình Home của SaleApp

Toolbar của Home được tùy biến bằng cách nhúng một RelativeLayout vào trong, cho phép tự định vị tên ứng dụng (txtLogo) bên trái và 2 icon (btnNotification, btnCart) bên phải bằng các thuộc tính layout_alignParentStart/End, thay vì sử dụng menu Toolbar chuẩn của Android cách làm này giúp kiểm soát chính xác hơn khoảng cách và kích thước icon theo đúng ý đồ thiết kế.

Danh mục sản phẩm hiển thị dạng cuộn ngang (rvCategories, LinearLayoutManager.HORIZONTAL) và lưới sản phẩm nổi bật (rvProducts, GridLayoutManager 2 cột) đều lấy dữ liệu thật trực tiếp từ AppDataStore.categoryCatalog và AppDataStore.productCatalog, không phải dữ liệu giả lập riêng cho màn hình này. Khi người dùng bấm vào một sản phẩm, mã productID thật được gửi kèm Intent sang ProductDetailActivity; khi quay lại Home (ví dụ sau khi thêm sản phẩm mới ở khu vực Admin), hàm onResume() gọi productAdapter.notifyDataSetChanged() để đồng bộ lại danh sách hiển thị đây chính là cơ chế “làm mới thủ công” đã đề cập ở Hình 3.1, do AppDataStore không có khả năng tự thông báo thay đổi (observable) như LiveData.

Ô tìm kiếm edtSearchHome bắt sự kiện bàn phím ảo IME_ACTION_SEARCH; khi người dùng nhập từ khoá và nhấn nút tìm kiếm trên bàn phím, từ khoá thật được gửi qua Intent (khóa SEARCH_QUERY) sang ProductListActivity để thực hiện lọc đây là tương tác dữ liệu thật, không phải minh hoạ tĩnh.

3.3.4. Giao diện Product List

[Chèn ảnh chụp màn hình thực tế tại đây lưới sản phẩm 2 cột, Toolbar có nút quay lại, hàng Chip lọc nhanh phía trên danh sách]

Hình 3.5. Giao diện màn hình Product List của SaleApp

Tiêu đề màn hình (txtTitle) được set động bằng một biểu thức when trong Kotlin, hiển thị một trong ba nội dung khác nhau tùy theo Intent nhận được: “Tìm kiếm: <từ khoá>” nếu có searchQuery, tên danh mục nếu có categoryName, hoặc “Sản phẩm” nếu không có cả hai cho thấy cùng một layout activity_product_list.xml được tái sử dụng cho nhiều ngữ cảnh điều hướng khác nhau (từ thanh tìm kiếm ở Home, hoặc từ việc bấm vào một danh mục).

Danh sách sản phẩm hiển thị là kết quả của một phép filter{} thật, thực hiện trực tiếp trên AppDataStore.productCatalog (kết hợp điều kiện khớp danh mục và khớp từ khoá, không phân biệt hoa thường nhờ ignoreCase = true), không phải một danh sách dựng sẵn. Tuy nhiên, bộ Chip lọc nhanh phía trên danh sách (chipPopular, chipPriceLowHigh, chipPriceHighLow) hiện chỉ là thành phần giao diện tĩnh: file Kotlin của Activity này không gán bất kỳ listener nào cho ChipGroup, do đó việc chọn các Chip này chưa có tác dụng sắp xếp lại danh sách như tên gọi của chúng thể hiện.

3.3.5. Giao diện Product Detail

[Chèn ảnh chụp màn hình thực tế tại đây ảnh lớn sản phẩm, nút quay lại nổi trên ảnh, tên/giá/mô tả sản phẩm, 2 nút cố định đáy “Thêm Giỏ Hàng” và “Mua Ngay”]

Hình 3.6. Giao diện màn hình Product Detail của SaleApp

Bộ cục sử dụng CoordinatorLayout kết hợp NestedScrollView để phân nội dung mô tả có thể cuộn, trong khi thanh hành động phía dưới (chứa 2 nút “Thêm Giỏ Hàng” và “Mua Ngay”) được đặt trong một MaterialCardView riêng, nằm ngoài vùng cuộn và có cardElevation cao (16dp), nhờ vậy luôn cố định ở đáy màn hình bất kể nội dung mô tả dài hay ngắn đúng yêu cầu thiết kế đã đặt ra ở mục 2.8.1(e). Nút quay lại (btnBackDetail) được đặt chồng trực tiếp lên ảnh sản phẩm lớn nhờ RelativeLayout, với nền là drawable circle_bg_white một kỹ thuật overlay đơn giản nhưng hiệu quả để nút luôn nổi rõ trên mọi loại ảnh nền.

Toàn bộ nội dung hiển thị (tên, giá, danh mục viết hoa, mô tả, số lượng đánh giá) đều được code gán lại bằng dữ liệu thật tra cứu qua AppDataStore.findProductById(), ghi đè lên các giá trị placeholder khai báo cứng trong XML (dùng để xem trước layout trong Android Studio). Một điểm cần lưu ý khi rà soát kỹ code: thanh đánh giá sao (RatingBar) trong layout được khai báo cứng android:rating="4.5" và không được tham chiếu lại trong

ProductDetailActivity.kt, nghĩa là RatingBar luôn hiển thị 4,5 sao cho mọi sản phẩm, không phản ánh đúng giá trị rating thật của từng sản phẩm trong AppDataStore (chỉ có số lượt đánh giá reviews được hiển thị đúng qua txtRatingCount).

3.3.6. Giao diện Cart

[Chèn ảnh chụp màn hình thực tế tại đây danh sách sản phẩm trong giỏ với bộ tăng/giảm số lượng, khối tổng tiền và nút “Tiến hành thanh toán” phía dưới]

Hình 3.7. Giao diện màn hình Cart của SaleApp

Đây là màn hình có logic giao diện động phức tạp nhất trong toàn ứng dụng. Bộ tăng/giảm số lượng trong item_cart.xml được tự dựng bằng 3 TextView (btnCartMinus, txtCartQuantity, btnCartPlus) xếp ngang trong một MaterialCardView bo tròn nhỏ, thay vì dùng một widget Stepper có sẵn cách làm này tuy tốn nhiều dòng XML hơn nhưng cho phép kiểm soát hoàn toàn về giao diện (màu sắc, khoảng cách, hiệu ứng).

Trong CartAdapter, khi bấm nút tăng/giảm, code cập nhật trực tiếp product.quantity và txtCartQuantity.text ngay tại Adapter, sau đó gọi callback onUpdateCart() để báo cho CartActivity tính lại tổng tiền một cách phân chia trách nhiệm hợp lý: Adapter chỉ chịu trách nhiệm cập nhật giao diện của item, còn Activity chịu trách nhiệm tổng hợp dữ liệu toàn giỏ hàng. Khi xóa sản phẩm, Adapter dùng đúng cặp notifyItemRemoved() và notifyItemRangeChanged() theo khuyến nghị của RecyclerView, thay vì notifyDataSetChanged() như đa số Adapter còn lại trong ứng dụng đây là điểm CartAdapter làm tốt hơn về mặt tối ưu hiệu năng hiển thị danh sách.

CartActivity.checkCartState() chuyển đổi visibility giữa layoutEmptyCart và cập (rvCartItems, cardPayment) hoàn toàn dựa vào việc AppDataStore.cartItems có rỗng hay không đây là một ví dụ điển hình của giao diện “hai trạng thái” được điều khiển trực tiếp bằng dữ liệu thật, không phải set cứng. Tuy nhiên, cũng cần lưu ý hai điểm chưa hoàn thiện: thứ nhất, CartAdapter không gán ảnh sản phẩm cho imgCartProduct (ảnh trong giỏ hàng luôn là placeholder_product mặc định, dù sản phẩm đã có ảnh thật được Admin chọn từ trước); thứ hai, ô “Mã giảm giá” (edtPromoCode, btnApplyPromo) và mục “Phí vận chuyển: Miễn phí” chỉ là thành phần giao diện tĩnh, chưa gắn logic xử lý nào trong CartActivity.kt.

3.3.7. Giao diện Checkout (demo thanh toán)

[Chèn ảnh chụp màn hình thực tế tại đây form thông tin giao hàng, lựa chọn phương thức thanh toán (COD/Chuyển khoản), khối tổng tiền và nút “Đặt Hàng” cố định đáy]

Hình 3.8. Giao diện màn hình Checkout của SaleApp

Màn hình được chia thành phần cuộn được (ScrollView chứa 2 MaterialCardView: thông tin giao hàng và phương thức thanh toán) và một MaterialCardView cố định ở đáy (cardOrderAction, cardElevation 8dp) hiển thị tổng tiền cùng nút “Đặt Hàng” cùng kỹ thuật cố định-đáy-màn-hình đã dùng ở Product Detail.

Đây là nơi dữ liệu nhập tay của người dùng thật sự được ghi vào hệ thống: CheckoutActivity.kt kiểm tra cả 3 trường tên, số điện thoại, địa chỉ không được để trống, kiểm tra giỏ hàng không được rỗng, sau đó gọi AppDataStore.placeOrder(customerName = name) để tạo đơn hàng mới và xoá giỏ hàng. Tuy nhiên, có một điểm chưa khớp giữa giao diện và dữ liệu được lưu lại: RadioGroup chọn phương thức thanh toán (rgPaymentMethods, gồm rbCOD và rbBank) hoàn toàn không được CheckoutActivity.kt đọc giá trị hay xử lý gì cả nghĩa là việc khách hàng chọn “Thanh toán khi nhận hàng” hay “Chuyển khoản” chỉ có ý nghĩa hiển thị, chưa được lưu vào đối tượng Order (data class Order hiện cũng chưa có trường paymentMethod). Tương tự, thông tin tên/số điện thoại/địa chỉ người nhận nhập ở bước này cũng không được lưu lại vào Order Order chỉ giữ lại customerName, chưa có trường địa chỉ hay số điện thoại giao hàng. Do giới hạn của đề tài, toàn bộ phần thanh toán cũng chỉ mang tính demo, không kết nối với bất kỳ công thanh toán thực tế nào.

3.3.8. Giao diện Order History

[Chèn ảnh chụp màn hình thực tế tại đây TabLayout 3 tab trạng thái phía trên, danh sách đơn hàng dạng card với mã đơn, tên khách, chip trạng thái màu]

Hình 3.9. Giao diện màn hình Order History của SaleApp

Phần TabLayout phân chia trạng thái (Tất cả / Đang xử lý / Đã hoàn thành) được dựng đúng theo thiết kế ở mục 2.8.1(h), nhưng OrderHistoryActivity.kt không gắn addOnTabSelectedListener nào cho tabLayoutHistory, nên việc chuyển tab hiện chưa có tác dụng lọc lại danh sách đơn hàng hiển thị bên dưới cả 3 tab tạm thời chỉ mang tính trang trí.

Đáng chú ý hơn, danh sách hiển thị ở màn hình này tái sử dụng đúng AdminOrderAdapter cùng một Adapter đang được dùng ở hai màn hình quản trị (AdminOrdersActivity, AdminDashboardActivity) gắn trực tiếp với toàn bộ AppDataStore.orders. Đây là một điểm vừa là ưu điểm vừa là hạn chế cần nêu rõ: ưu điểm là tận dụng lại được component đã viết, tránh trùng lặp code; hạn chế là vì ứng dụng chưa có khái niệm “người dùng đang đăng nhập”, màn hình “Đơn hàng của tôi” trên thực tế đang hiển thị toàn bộ đơn hàng của mọi khách hàng trong hệ thống (bao gồm cả đơn của khách khác), thay vì chỉ lọc theo đúng khách hàng hiện tại như tên màn hình thể hiện.

3.3.9. Giao diện Profile

[Chèn ảnh chụp màn hình thực tế tại đây phần đầu trang nền màu primary với ảnh đại diện tròn, tên/email, card “Quản lý tài khoản”, nút chuyển Admin và nút đăng xuất]

Hình 3.10. Giao diện màn hình Profile của SaleApp

Phần đầu trang (rlProfileHeader) dùng nền màu primary đặc, chứa ảnh đại diện dạng tròn (ShapeableImageView với shapeAppearanceOverlay bo 50%) cùng tên và email người dùng. Tuy nhiên, hai TextView txtUserName và txtUserEmail đang hiển thị giá trị khai báo cứng ngay trong file activity_profile.xml (“Nguyễn Văn A”, “vana@gmail.com”) và hoàn toàn không được ProfileActivity.kt cập nhật lại bằng dữ liệu thật hệ quả trực tiếp của việc ứng dụng chưa có cơ chế lưu lại “người dùng nào đang đăng nhập” sau bước LoginActivity.

Ba mục trong card “Quản lý tài khoản” (Chỉnh sửa hồ sơ, Đổi mật khẩu, Lịch sử đơn hàng) được dựng bằng RelativeLayout chứa TextView tiêu đề và ImageView mũi tên (ic_chevron_right) một mẫu UI quen thuộc cho danh sách menu dạng dòng. Trong đó, “Lịch sử đơn hàng” điều hướng thật sang OrderHistoryActivity; hai mục còn lại hiện chỉ hiển thị Toast “đang phát triển” khi bấm. Nút “Chuyển sang giao diện Quản trị” điều hướng trực tiếp sang

AdminDashboardActivity mà không qua bất kỳ kiểm tra quyền nào, còn nút “Đăng xuất” điều hướng về LoginActivity kèm cờ FLAG_ACTIVITY_NEW_TASK và FLAG_ACTIVITY_CLEAR_TASK để xóa sạch ngăn xếp Activity phía sau đảm bảo người dùng không thể bấm nút Back để quay lại các màn hình cần đăng nhập sau khi đã đăng xuất.

3.3.10. Giao diện Notifications

[Chèn ảnh chụp màn hình thực tế tại đây Toolbar tiêu đề “Thông báo mới nhất” và danh sách thông báo (lưu ý: cần chụp đúng trạng thái thực tế đang chạy của màn hình này xem phân tích bên dưới)]

Hình 3.11. Giao diện màn hình Notifications của SaleApp

Màn hình Thông báo có layout hoàn chỉnh: Toolbar tiêu đề và một RecyclerView (rvNotifications) đã được chuẩn bị sẵn item layout item_notification.xml (gồm tiêu đề, thời gian, nội dung thông báo, có cả dữ liệu mẫu xem trước dạng tools:text ngay trong layout để dễ thiết kế). Tuy nhiên, khi rà soát NotificationsActivity.kt, Activity này chỉ thực hiện duy nhất việc inflate ViewBinding và gọi setContentView hoàn toàn không gán Adapter nào cho rvNotifications. Hệ quả là khi chạy thực tế, màn hình Thông báo sẽ hiển thị trống (không có nội dung nào) dù về mặt giao diện đã được thiết kế đầy đủ. Đây là một ví dụ rõ về sự khác biệt giữa “màn hình đã có layout” và “màn hình đã có dữ liệu thật” hai khái niệm cần được phân biệt rõ khi đánh giá mức độ hoàn thành của một ứng dụng, và cũng là lý do hình ảnh thực tế chèn ở Hình 3.11 cần được chụp đúng trạng thái màn hình trống hiện tại, tránh gây hiểu lầm về mức độ hoàn thiện.

3.3.11. Giao diện khu vực Admin

[Chèn ảnh chụp màn hình thực tế tại đây nên ghép nhiều ảnh chụp màn hình nhỏ theo dạng lưới (collage) để thể hiện đủ 6 màn hình con trong cùng một hình minh họa tổng thể]

Hình 3.12. Giao diện khu vực Admin của SaleApp (Dashboard, Quản lý sản phẩm, Thêm/Sửa sản phẩm, Quản lý đơn hàng, Quản lý người dùng, Báo cáo)

Khu vực Admin gồm 6 màn hình con, được trình bày chung trong một mục vì cùng chia sẻ một phong cách thiết kế nhất quán (nền admin_bg #F3F4F6, Toolbar trắng, MaterialCardView bo góc 12–16dp) và cùng thao tác trên một nguồn dữ liệu duy nhất là AppDataStore.

a) Dashboard (Bảng điều khiển chính): Màn hình hiển thị 2 thẻ KPI (Doanh thu, Đơn đặt hàng) ở đầu trang và 4 thẻ truy cập nhanh (Đơn hàng, Kho hàng, Khách hàng, Báo cáo) dựng bằng GridLayout 2 cột, mỗi thẻ là một MaterialCardView có id riêng (cardGoOrders, cardGoProducts, cardGoUsers, cardGoReports) được gắn setOnClickListener trực tiếp trong AdminDashboardActivity.kt cách làm phù hợp vì số lượng thẻ cố định, không cần thiết phải dựng cả một RecyclerView chỉ cho 4 phần tử. Tuy nhiên, hai chỉ số KPI (txtKpiRevenue hiển thị “36.480K”, txtKpiOrders hiển thị “3 đơn”) là giá trị khai báo cứng trực tiếp trong activity_admin_dashboard.xml; AdminDashboardActivity.kt không hề tham chiếu đến hai TextView này, nên các số liệu này không được tính toán động từ AppDataStore.orders như tên gọi “Bảng điều khiển chính” gợi ý, mà giữ nguyên giá trị tĩnh bất kể dữ liệu đơn hàng thực tế thay đổi ra sao. Phần danh sách “Yêu cầu duyệt đơn gần đây” (rvRecentOrdersAdmin) thì có dữ liệu thật, tái sử dụng đúng AdminOrderAdapter đã phân tích ở mục 3.3.8.

b) Quản lý sản phẩm và Thêm/Sửa sản phẩm: AdminProductsActivity hiển thị danh sách sản phẩm thật từ AppDataStore.productCatalog qua AdminProductAdapter, kèm FloatingActionButton (fabAddProduct) ở góc dưới phải để mở EditProductActivity. Một chi tiết thiết kế đáng chú ý: nút mở màn hình thêm mới không gửi kèm Intent extra PRODUCT_ID nào, trong khi nút “Sửa” ở từng item lại gửi kèm PRODUCT_ID EditProductActivity.kt dựa vào chính sự có-mặt-hay-không của extra này (productId != null) để quyết định đang ở chế độ Thêm mới hay Sửa, từ đó quyết định có gọi editingProduct?.let{...} để đổ sẵn dữ liệu cũ vào form hay không. Đây là cách tái sử dụng một layout/Activity duy nhất cho hai mục đích khác nhau khá gọn gàng. Form thêm/sửa có ô chọn ảnh thật từ thư viện máy (PickVisualMediaRequest), xem trước ảnh ngay khi chọn nhờ hàm mở rộng setProductImage() dùng chung cho toàn ứng dụng, và validate đầy đủ trước khi lưu (tên, danh mục phải thuộc danh sách hợp lệ, giá và số lượng phải là số). Đáng lưu ý, nút back trên Toolbar (toolbarEditProduct) có khai báo app:navigationIcon trong XML nhưng EditProductActivity.kt không gọi setNavigationOnClickListener, nên icon mũi tên quay lại hiển thị đúng nhưng bấm vào không có tác dụng người dùng phải dùng nút Back hệ thống của thiết bị. Ngoài ra,

AdminProductAdapter chỉ có nút “Sửa” (btnAdminEditProduct), chưa có chức năng xóa sản phẩm như đã đề cập ở thiết kế mục 2.8.2(b).

c) Quản lý đơn hàng: AdminOrdersActivity hiển thị danh sách đơn hàng thật qua AdminOrderAdapter, nhưng khi bấm vào một đơn, ứng dụng chỉ hiển thị Toast “Xem chi tiết đơn: <mã đơn>” chưa có màn hình chi tiết đơn hàng hay cơ chế cập nhật trạng thái qua dropdown như đã thiết kế ở mục 2.8.2(d). Trên thực tế, AppDataStore hiện cũng chưa có hàm nào để thay đổi status của một Order sau khi đã tạo; mọi đơn hàng mới đặt đều cố định ở trạng thái “Chờ duyệt”, chỉ riêng đơn hàng mẫu DH-8924 được khai báo sẵn ở trạng thái “Đã giao” mới có màu nền khác (nhờ logic đổi màu theo status trong AdminOrderAdapter).

d) Quản lý người dùng: AdminUsersActivity là màn hình đơn giản nhất trong khu vực Admin: chỉ một Toolbar và một RecyclerView hiển thị 3 người dùng mẫu thật từ AppDataStore.users qua AdminUserAdapter. Tương tác click hiện chỉ dừng ở mức Toast hiển thị tên người dùng, chưa có màn hình chi tiết hay chức năng khoá/mở khoá tài khoản như mô tả trong thiết kế.

e) Báo cáo thống kê: Đây là màn hình tĩnh hoàn toàn về mặt dữ liệu trong số các màn hình Admin. AdminReportsActivity.kt chỉ thực hiện inflate ViewBinding, không có bất kỳ dòng code xử lý nào khác. Thẻ “Thống kê doanh số tháng” chỉ là một View đơn (barChartSales) tô nền xám phẳng #EEF2F6, đóng vai trò placeholder cho một biểu đồ cột (dự kiến dùng thư viện như MPAndroidChart) chứ chưa thực sự vẽ biểu đồ nào. Thẻ “Sản phẩm bán chạy nhất” chứa một TableLayout nhưng mới chỉ có duy nhất dòng tiêu đề (Tên sản phẩm / Doanh số), chưa có dòng dữ liệu nào được thêm vào bằng code nghĩa là về bản chất, màn hình Báo cáo hiện tại là một khung giao diện đã chuẩn bị sẵn vị trí, chưa có logic tính toán hay truy vấn dữ liệu nào chạy phía sau.

Nhìn chung, khu vực Admin thể hiện rõ một thực tế khá điển hình khi xây dựng ứng dụng trong thời gian có hạn: các luồng có tương tác dữ liệu trực tiếp và đơn giản (xem danh sách, thêm/sửa sản phẩm) được hoàn thiện khá đầy đủ, trong khi các chức năng đòi hỏi xử lý nghiệp vụ phức tạp hơn hoặc tổng hợp/tính toán số liệu (cập nhật trạng thái đơn hàng, thống kê báo cáo) mới chỉ hoàn thành phần giao diện, chưa có logic thật phía sau.

3.4. Đánh giá kết quả đạt được

3.4.1. Các chức năng/giao diện đã hoàn thành

Sau quá trình xây dựng, ứng dụng SaleApp đã hoàn thành các nội dung sau:

- Thiết kế và hoàn thiện đầy đủ 16 màn hình giao diện theo phong cách Material 3 nhất quán, bao gồm cả luồng khách hàng và khu vực Admin.

- Đăng nhập và đăng ký với validate dữ liệu nhập đầy đủ (dù chưa đối chiếu/lưu vào dữ liệu hệ thống thật, sẽ nêu ở mục 3.4.3).
- Xem danh sách sản phẩm theo trang chủ, theo danh mục và theo từ khoá tìm kiếm; xem chi tiết sản phẩm với dữ liệu thật.
- Thêm sản phẩm vào giỏ hàng, tăng/giảm số lượng, xoá sản phẩm, tự động tính lại tổng tiền toàn bộ vận hành đúng logic trên dữ liệu thật trong bộ nhớ.
- Đặt hàng thật (tạo Order mới, sinh mã đơn tự động tăng dần, xoá giỏ hàng sau khi đặt) và xem lại lịch sử đơn hàng.
- Quản lý sản phẩm phía Admin: thêm mới/chỉnh sửa sản phẩm với đầy đủ validate và chọn ảnh thật từ thiết bị; dữ liệu cập nhật ngay lập tức và phản ánh đúng ở các màn hình khách hàng nhờ cơ chế dữ liệu tập trung AppDataStore.
- Xem danh sách đơn hàng và danh sách khách hàng phía Admin với dữ liệu thật.

3.4.2. Ưu điểm của hệ thống

- Giao diện trực quan, nhất quán về phong cách (màu sắc, bo góc, elevation) giữa các màn hình; các bước thao tác từ xem sản phẩm đến đặt hàng đơn giản, dễ làm theo.
- Tốc độ phản hồi rất nhanh vì toàn bộ dữ liệu được giữ trong bộ nhớ (in-memory), không có độ trễ truy vấn cơ sở dữ liệu hay chờ tải qua mạng phù hợp để trình diễn (demo) trực tiếp.
- Cấu trúc code tách rõ giữa giao diện (XML) và xử lý logic (Kotlin) thông qua View Binding áp dụng nhất quán ở toàn bộ 16 Activity và 6 Adapter, hạn chế tối đa lỗi liên quan đến tham chiếu sai view.
- Việc tập trung toàn bộ dữ liệu vào một nguồn duy nhất (AppDataStore) giúp các thay đổi ở một màn hình (ví dụ thêm sản phẩm ở Admin) phản ánh ngay tại các màn hình khác (Home, Product List) mà không cần cơ chế đồng bộ phức tạp, dù phải gọi `notifyDataSetChanged()` thủ công ở `onResume()`.
- Một số chi tiết kỹ thuật được xử lý cẩn thận, ví dụ dùng `product.copy()` khi thêm vào giỏ hàng để tránh lỗi tham chiếu chung vùng nhớ, hay xử lý ảnh sản phẩm an toàn (`setProductImage()` có try-catch, tự động về placeholder khi URI lỗi/rỗng), giúp ứng dụng không bị crash trong các trường hợp dữ liệu bất thường.
- Tái sử dụng component hợp lý (AdminOrderAdapter dùng chung cho 3 màn hình khác nhau), giảm trùng lặp code.

3.4.3. Hạn chế của hệ thống

- Dữ liệu chưa được lưu trữ lâu dài: toàn bộ chạy trong bộ nhớ (in-memory), mất hoàn toàn ngay khi tắt ứng dụng hoặc tiến trình bị hệ điều hành thu hồi, không chỉ khi gỡ ứng dụng khác biệt lớn so với thiết kế SQLite ban đầu ở Chương 2 (đã phân tích kỹ ở mục 3.2.5).
- Chưa có cơ chế đăng nhập và phân quyền thật: mọi tài khoản đăng nhập đều vào cùng một giao diện, không phân biệt User/Admin; bất kỳ ai cũng truy cập được khu vực Admin từ màn hình Profile.
- Đăng ký tài khoản mới chưa được lưu vào hệ thống; màn hình Profile hiển thị thông tin cố định, không gắn với người dùng đang thực sự sử dụng.
- Một số màn hình/thành phần đã có giao diện hoàn chỉnh nhưng chưa gắn dữ liệu hoặc logic thật: màn hình Thông báo (chưa gắn Adapter), màn hình Báo cáo (biểu đồ và bảng thống kê đều tĩnh), chỉ số KPI ở Dashboard (giá trị khai báo cứng), bộ Chip lọc giá ở Product List, mã giảm giá ở Cart, TabLayout trạng thái ở Order History, RadioGroup phương thức thanh toán ở Checkout.
- Đơn hàng (Order) chưa lưu đầy đủ thông tin người nhận (địa chỉ, số điện thoại) và phương thức thanh toán đã chọn; chưa có cơ chế cập nhật trạng thái đơn hàng sau khi tạo.
- Lịch sử đơn hàng và danh sách đơn ở Admin hiện hiển thị chung toàn bộ AppDataStore.orders, chưa lọc theo từng khách hàng cụ thể do chưa có khái niệm phiên đăng nhập (session).
- Một vài chi tiết hiển thị chưa khớp hoàn toàn với dữ liệu thật, ví dụ RatingBar ở màn Chi tiết sản phẩm luôn cố định 4,5 sao, ảnh sản phẩm trong giỏ hàng không được cập nhật theo ảnh thật của sản phẩm.
- Kiến trúc code dùng chung một object Singleton truy cập trực tiếp từ Activity, chưa áp dụng mô hình phân tầng như MVVM/Repository; phần lớn Adapter cập nhật bằng notifyDataSetChanged() thay vì DiffUtil/ListAdapter, có thể ảnh hưởng đến hiệu năng khi danh sách dữ liệu lớn hơn.
- Chưa có Unit Test/Instrumented Test để kiểm thử tự động các hàm xử lý dữ liệu trong AppDataStore.

Toàn bộ chuỗi văn bản hiển thị được viết trực tiếp (hardcode) trong Kotlin/XML, hầu như không sử dụng strings.xml, gây khó khăn nếu muốn đa ngôn ngữ hoá hoặc chỉnh sửa nội dung tập trung về sau.

KẾT LUẬN

Kết quả đạt được

Qua quá trình thực hiện đề tài, em đã hoàn thành việc xây dựng ứng dụng bán hàng online SaleApp trên nền tảng Android, sử dụng Kotlin để xử lý logic, XML để thiết kế giao diện và SQLite để lưu trữ dữ liệu cục bộ. Ứng dụng mô phỏng đầy đủ quy trình mua sắm cơ bản: từ đăng ký, đăng nhập, xem sản phẩm, thêm vào giỏ hàng, đặt hàng, đến quản lý sản phẩm và đơn hàng ở phía Admin. Bên cạnh sản phẩm phần mềm, em cũng nắm vững hơn quy trình phân tích và thiết kế hệ thống thông qua việc vẽ Use Case, Class Diagram, Sequence Diagram và thiết kế cơ sở dữ liệu trước khi bắt tay vào code.

Khó khăn gặp phải

Trong quá trình làm đồ án, em gặp một số khó khăn nhất định. Về thiết kế giao diện, do chưa có nhiều kinh nghiệm nên ở giai đoạn đầu em mất khá nhiều thời gian để canh chỉnh layout cho cân đối giữa các loại màn hình khác nhau, nhất là khi chuyển từ wireframe sang code XML thật. Về xử lý dữ liệu, việc đồng bộ dữ liệu giữa các màn hình ví dụ khi thêm sản phẩm vào giỏ hàng ở một Activity nhưng cần cập nhật lại tổng tiền hiển thị ở Activity khác cũng khiến em gặp một vài lỗi liên quan đến việc dữ liệu không cập nhật kịp thời. Ngoài ra, việc làm quen với Kotlin sau khi đã quen thuộc với cú pháp Java cũng cần thêm thời gian, đặc biệt là phần xử lý null safety.

Hướng phát triển tương lai

Nếu có điều kiện phát triển thêm, em dự định mở rộng đề tài theo các hướng sau:

- Tích hợp cổng thanh toán thật (ví dụ VNPay, Momo) để thay thế phần thanh toán mô phỏng hiện tại.
- Đưa dữ liệu lên server hoặc dịch vụ backend như Firebase, giúp dữ liệu được đồng bộ giữa nhiều thiết bị và không bị mất khi gỡ ứng dụng.
- Bổ sung chức năng tìm kiếm và lọc sản phẩm theo nhiều tiêu chí hơn (tên, mức giá, đánh giá).
- Thêm thông báo đẩy (push notification) để báo cho người dùng khi trạng thái đơn hàng thay đổi.
- Tối ưu lại giao diện để hiển thị tốt hơn trên nhiều kích cỡ màn hình, bao gồm cả máy tính bảng.

Nhìn lại toàn bộ quá trình, đề tài này không chỉ giúp em hoàn thành một bài tập lớn mà còn là cơ hội để em thực hành lại từ đầu đến cuối một quy trình phát triển ứng dụng di động, từ

phân tích, thiết kế, đến lập trình và kiểm thử một trải nghiệm mà em nghĩ sẽ còn hữu ích cho các môn học và đồ án sau này.